Tracking the Evolution of Nested Learning

A RAG-Based Question-Answering System over the Nested Learning Research Corpus

NLP with Deep Learning
Assignment 3 Mini-Project 1

Abdullah Irfan	29266
Ameer Hamza	28308
Arhum Ali	29288

Live System Access
https://rockybhai0159-tracking-the-evolution-of-nested-l-acb77e7.hf.space/

ABSTRACT

This report presents Tracking the Evolution of Nested Learning, a Retrieval-Augmented Generation (RAG) system designed to answer complex, cross-paper queries over a curated corpus of 15 academic papers on the Nested Learning (NL) paradigm and its surrounding theoretical ecosystem. The system goes substantially beyond basic semantic search, implementing a four-stage hybrid retrieval pipeline combining BM25 keyword search, dense semantic retrieval via Pinecone, Reciprocal Rank Fusion (RRF), and Cross-Encoder reranking, followed by answer generation using GPT-4o-mini or Llama-3.1-8B-Instant, and fully automated evaluation via an LLM-as-a-Judge pipeline measuring Faithfulness and Relevancy.

Figure 1 illustrates the full system construction pipeline, spanning ve stages executed on Kaggle (T4 GPU): (1) Preprocessing 15 PDFs were extracted and cleaned using PyMuPDF; (2) Chunking three strategies (Fixed, Recursive, Semantic) were applied, producing 2,007, 2,753, and 1,455 chunks respectively; (3) Embedding and Pinecone Upsert all chunks were embedded using text-embedding-3-small (1,536 dimensions) and loaded into three separate Pinecone indexes; (4) Golden Set Generation 30 high-quality technical question answer pairs were generated using Llama-3.3-70B-Versatile; and (5) BM25 Index Construction three BM25 indexes were built in-memory, one per chunking strategy.

Figure 2 illustrates the evaluation and ablation pipeline (Part 6). For each of the 30 golden questions, the system simultaneously executes a semantic Pinecone search (top 15) and a BM25 keyword search (top 15), merges both candidate sets via RRF, and reranks the merged pool using a Cross-Encoder to select the nal top 5 context chunks. These chunks are passed to the generation model with a custom prompt; the generated answer is then evaluated by an LLM judge using claim extraction and veri cation (Faithfulness) and surrogate question generation (Relevancy). A comprehensive ablation study across 19 con gurations identi es Recursive chunking + Hybrid retrieval + GPT-4o-mini (Con guration C5) as the best overall system, achieving a Faithfulness score of 0.982 and Relevancy of 0.709. The system is deployed as a live web application on Hugging Face Spaces.

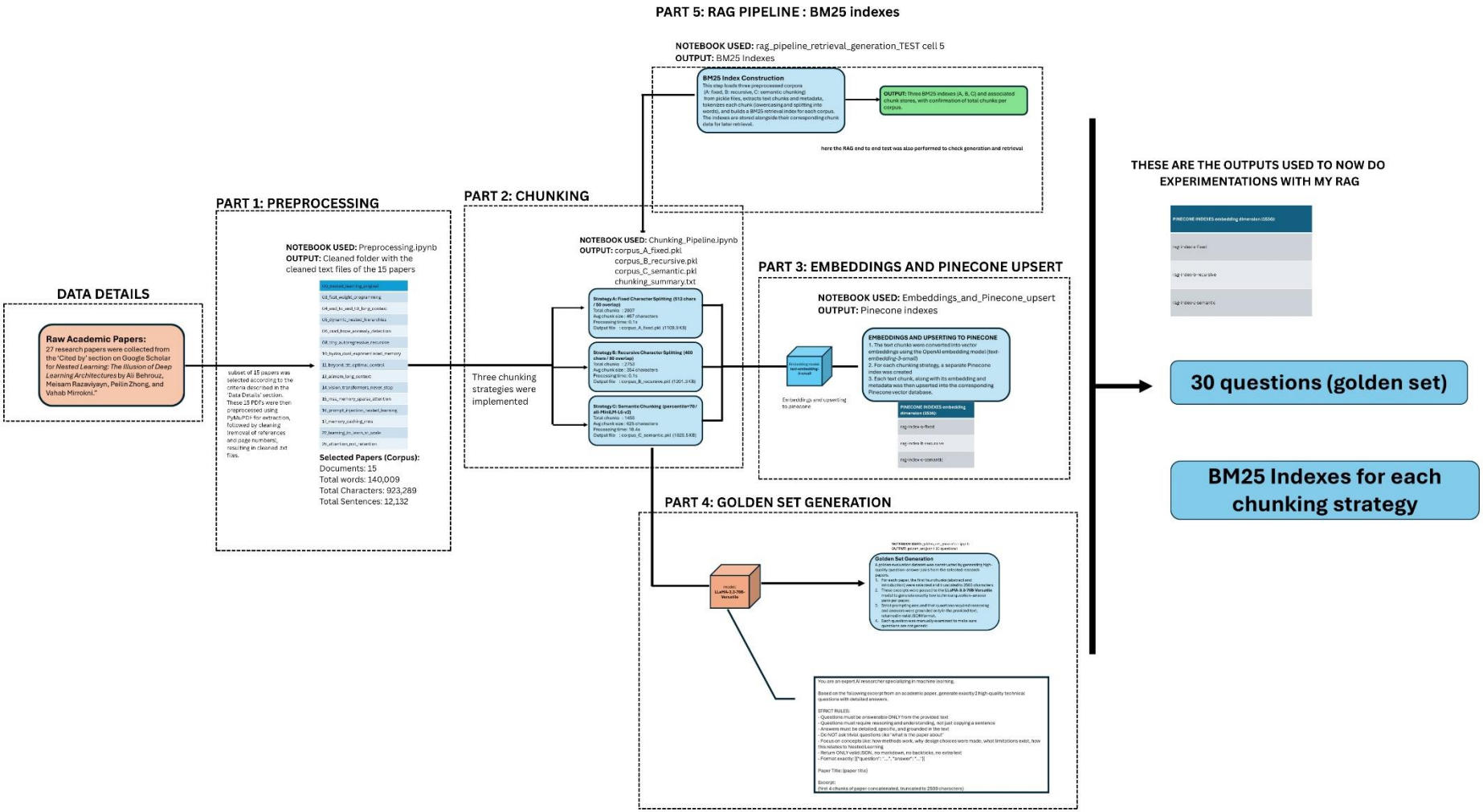

Figure 1: Full RAG system construction pipeline (Parts 1 5): Preprocessing, Chunking (three strategies), Embedding and Pinecone Upsert, Golden Set Generation, and BM25 Index Construction. All stages were executed on Kaggle with a T4 GPU.

PART 6: Evaluation + Ablation Study

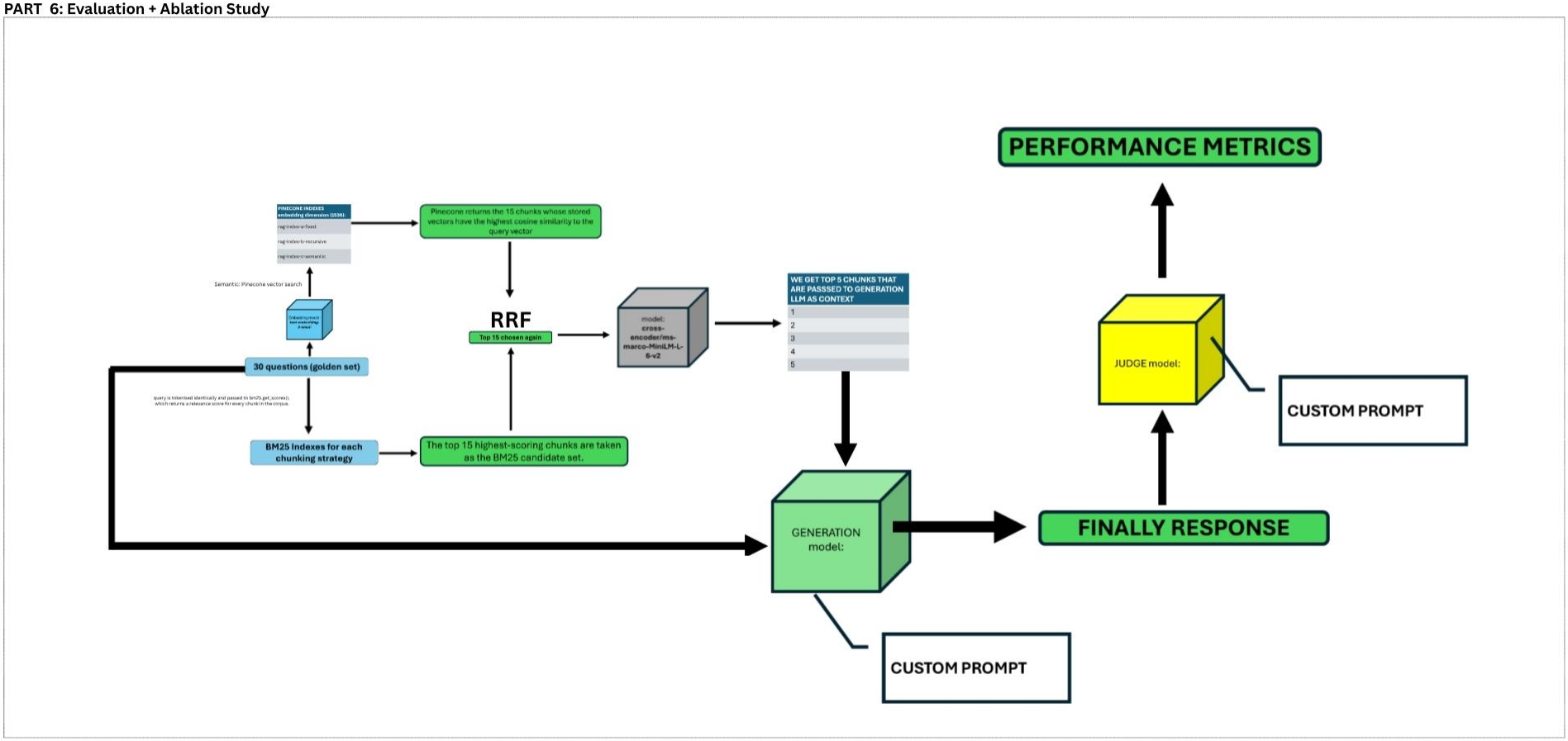

Figure 2: Evaluation and ablation pipeline (Part 6): a 30-question golden set is processed through dual-path retrieval (Pinecone semantic + BM25), merged via RRF, reranked by a Cross-Encoder, and fed to the generation model. The nal response is evaluated by a custom-prompted LLM judge measuring Faithfulness and Relevancy to produce the performance metrics reported in Section 4.

CONTENTS

1. **Platform Details**

2. **Data Details**
   - 2.1 The Why Behind Choosing This Field (Dataset)
   - 2.2 Dataset Source and Collection
   - 2.3 Curation, Selection, and Corpus Statistics
   - 2.4 Preprocessing (Preprocessing.ipynb)

3. **Algorithms, Models, and Retrieval Methods**
   - 3.1 Chunking Strategies
   - 3.2 Retrieval Methods
   - 3.3 All Models Used Throughout the System
   - 3.4 Prompt Structure
   - 3.5 All Configurations Tested

4. **Performance Metrics**
   - 4.1 Master Results Table – All Configurations
   - 4.2 Chunking Strategy Comparison
     - 4.2.1 Semantic-Only Retrieval (C1–C3)
     - 4.2.2 Hybrid + Rerank Retrieval (C4–C6)
   - 4.3 Retrieval Mode: Semantic vs Hybrid + Rerank
   - 4.4 Generator Model Comparison
   - 4.5 Latency and Computational Efficiency
   - 4.6 Prompt Engineering Ablation (P1–P4)
   - 4.7 Cross-Dimensional Analysis and Key Findings
     - 4.7.1 Faithfulness vs Relevancy
     - 4.7.2 Faithfulness Variance
     - 4.7.3 Chunking as Dominant Driver
     - 4.7.4 Interaction Effects
     - 4.7.5 Cost–Quality Trade-off
   - 4.8 Example Queries and Verification Results
     - 4.8.1 Query 1: Nested Learning and In-Context Learning
     - 4.8.2 Query 2: Nested Optimization Problems and Context Flow
     - 4.8.3 Query 3: Fast Weight Programmers vs Conventional RNNs

5. **Best Model Selection**
   - 5.1 Selection Framework
   - 5.2 Primary Best Configuration (C5)
   - 5.3 Enhanced Best Configuration (P2)
   - 5.4 Speed-Optimized Alternative (C11)
   - 5.5 Summary Recommendation

6. **Reproducibility**
    6.1 Hardware and Platform Requirements
    6.2 Corpus Collection
    6.3 Preprocessing Steps
    6.4 Chunking Configuration
    6.5 Embedding and Pinecone Setup
    6.6 BM25 Index Construction
    6.7 Golden Set Generation
    6.8 Retrieval and Generation Pipeline
    6.9 Evaluation Pipeline

References

Platform Details

All computational stages of this project were executed across two platforms.

KAGGLE (Primary Computation Platform)

All 6 Jupyter notebooks comprising the system were run on Kaggle, using a T4 GPU accelerator (16 GB VRAM). Kaggle was chosen as the sole computation environment for its reproducibility guarantees: sessions are isolated, package environments are pinnable, and notebooks can be re-run deterministically from a clean state. The T4 GPU was required specially for Strategy C: Semantic Chunking, which invokes the all-MiniLM-L6-v2 sentence-transformer model locally during the chunking phase to compute per-sentence embeddings. Without GPU acceleration, the semantic chunking stage would take prohibitively long on 428 pages of academic text.

HUGGIN FACE SPACES (DEPLOYMENT)

The live web application is deployed on Hugging Face Spaces using a Gradio interface. The Space is publicly accessible and allows users to input a query and receive the generated answer, retrieved context chunks, Faithfulness and Relevancy scores, and source citations in real time. The deployment uses the best-performing pipeline con guration (Con guration C5: Recursive chunking, Hybrid retrieval, GPT-4o-mini generation) as the production backend. The Pinecone index is cloud-hosted, and query embeddings are computed on-app using text-embedding-3-small via the OpenAI API, avoiding any local index storage on the Space.

Table 2: Platform assignments by pipeline stage.

Pipeline Stage	Platform	Hardware
PDF preprocessing and cleaning	Kaggle	T4 GPU
Chunking (Fixed, Recursive, Semantic)	Kaggle	T4 GPU
Document embedding + Pinecone upsert	Kaggle	T4 GPU
Golden set generation	Kaggle	T4 GPU
Retrieval, generation (end to end test)	Kaggle	T4 GPU
Evaluation and ablation	Kaggle	T4 GPU
Live web application	Hugging Face Spaces	CPU (cloud)
Vector database storage	Pinecone (cloud)	

2. Data Details

2.1 The why behind choosing this field (dataset):

The RAG system built for this project is titled *Tracking the Evolution of Nested Learning*. The choice of Nested Learning (NL) as the knowledge domain was deliberate and motivated by several converging factors.

A paradigm at the frontier, not yet settled. Nested Learning was introduced in December 2025 at NeurIPS — one of the most competitive venues in machine learning — making it a genuinely emerging paradigm at the time of corpus construction. Unlike mature fields where textbook-level answers are retrievable with a simple keyword search, NL exists in an active, contested theoretical space. Its core claim — that architectures as diverse as Transformers, RNNs, and state-space models are all special cases of a unified multi-level optimization framework — has attracted both extensions and scrutiny within months of publication. A RAG system over this corpus must handle nuanced, cross-paper reasoning: not just "what is Nested Learning?" but "how does paper X challenge or extend NL's memory abstraction?" This is precisely the kind of query where retrieval quality matters and where a well-tuned RAG pipeline demonstrates clear value over a naïve keyword search.

A natural citation network ideal for RAG. The anchor paper had already accumulated 27 citing papers at the time of collection, spanning Core Extensions, Applications, and Related Theory. This citation network is a structural gift for a RAG corpus: each citing paper engages with NL from a distinct angle, meaning the corpus covers a shared conceptual vocabulary from multiple perspectives. A query about, say, NL's test-time memorization mechanism can be answered from the anchor paper itself, corroborated by Memory Caching (Paper #17), challenged by the attention-vs-retention framing of Paper #26, and grounded neurobiologically by Paper #3. No single document contains all of this — multi-document retrieval is not just helpful but necessary, which is the ideal stress-test for a RAG pipeline.

Controlled corpus size with high information density. With 15 selected papers totalling approximately 428 pages and an estimated 600 chunks, the corpus is large enough to require genuine retrieval — answers cannot be brute-forced from a single document — yet small enough for all configurations to be evaluated exhaustively within the project's scope. The curation process (described below) further ensured that every included paper carries substantial NL engagement rather than superficial background citations, maximising the information-per-chunk density that is critical for faithful answer generation.

Personal intellectual investment. NL's central thesis — that the apparent diversity of deep learning architectures collapses into a single nested optimization schema — is one of the more intellectually provocative claims in recent ML literature. Building a RAG system around it meant engaging deeply with the paper's mathematical framework and with the community's evolving response to it. This made the project a richer learning experience than a domain chosen purely for convenience: understanding the corpus well enough to design good evaluation questions required genuinely grasping what NL claims, what it unifies, and where its boundaries lie.

2.2 Dataset Source and Collection:

The data for this project was collected through the following methodology

The corpus for this system was constructed entirely from academic literature in the domain of **Nested Learning (NL)** and its surrounding theoretical space.

The anchor document is the original paper — ***Nested Learning: The Illusion of Deep Learning Architectures*** **by Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni**

Behrouz, A., Razaviyayn, M., Zhong, P., Mirrokni, V.: Nested learning: The illusion of deep learning architectures. In: The Thirty-ninth Annual Conference on Neural Information Processing Systems (2025)

Link to paper: https://arxiv.org/pdf/2512.24695

To build the corpus of citing papers, Google Scholar's "Cited by" feature was used on the original NL paper to retrieve all known citations at the time of collection. This yielded 27 candidate papers (there were 29 but two of them were not in English).

Then I downloaded all the papers, made an excel sheet to track the paper title, its authors, its publishing date, link to the pdf and the number of pages per pdf.

2.3 Curation, Selection and Corpus Statistics:

Then I gave the excel sheet to claude sonnet 4.5 to evaluate each candidate across five criteria:

1. relevance to NL's core concepts,
2. conceptual depth,
3. contribution type,
4. diversity relative to already-selected papers,
5. information density.

Papers were classified into four categories —
Core Extension, Application, Related Theory, and Weak Citation — and scored on a 1–5 relevance scale.
The results were as follows:

Corpus Summary

Total Papers	Selected	Excluded	Total Pages	Est. Chunks	Avg Score
28	15 papers	13 papers	~428 pages	~600 chunks	3.4 / 5.0

Colour Legend

Core Extension	Application	Related Theory	Weak Citation	SELECTED	EXCLUDED

Full Paper Inventory (28 papers evaluated)

#	Paper Title	Authors	Year	Pages	Category	Score	Status	Justification / Reason	PDF
SECTION A — SELECTED PAPERS (15 papers included in the final corpus)									
#0	Nested Learning: The Illusion of Deep Learning Architectures	*Behrouz, Razaviyayn, Zhong, Mirrokni*	2025	52	**Core Extension**	●●●●● 5/5	SELECTED	Anchor paper. Introduces NL as nested multi-level optimization with context flows. Entire corpus is built around this work.	PDF
#17	Memory Caching: RNNs with Growing Memory	*Behrouz, Li, Deng, Zhong, Razaviyayn, Mirrokni*	2026	22	**Core Extension**	●●●●● 5/5	SELECTED	Same lab as NL authors. Explicitly grounds Memory Caching in NL's test-time memorization perspective. High information density.	PDF
#10	HYDRA: Dual Exponentiated Memory for Multivariate Time Series Analysis	*Meskin, Mirrokni, Najar, Behrouz*	2025	22	**Core Extension**	●●●●● 5/5	SELECTED	Co-authored by Behrouz. Designs a meta in-context memory module — a direct application of NL's memorization principle to time series.	PDF
#11	Beyond Test-Time Training: Learning to Reason via Hardware-Efficient Optimal Control	*Wang, Yang, Wang, Xiao, Liu, Yu, Lou, Li, Wang, Lin,...*	2026	30	**Core Extension**	●●●●● 5/5	SELECTED	Introduces TTC layer using a nested objective for planning before prediction. Abstract explicitly references NL's nested structure.	PDF
#4	End-to-End Test-Time Training for Long Context	*Tandon, Dalal, Li, Koceja, Roed, Buchanan, Wang, Les...*	2025	29	**Core Extension**	●●●●○ 4/5	SELECTED	Directly instantiates NL's in-context memorization loop for long-context tasks. Provides experimental evidence for NL's continual learning claims.	PDF
#5	Dynamic Nested Hierarchies: Pioneering Self-Evolution in ML Architectures for Lifelong Intelligence	*Akbar Anbar Jafari, Ozcinar, Anbarjafari*	2025	12	**Core Extension**	●●●●○ 4/5	SELECTED	Title directly maps to NL's nested structure and lifelong learning thesis. Proposes self-evolving architectures through dynamic hierarchies.	PDF
#6	CRAD-HOPE: Brain-inspired Nested Learning Framework for Few-Shot Anomaly Detection	*Cao, Cao, Wu*	2026	30	**Application**	●●●●○ 4/5	SELECTED	Only paper that names a complete system a 'nested learning framework'. Applies NL concepts to anomaly detection — a key application case study.	PDF
#3	Fast Weight Programming and Linear Transformers: From Machine Learning to Neurobiology	*Irie, Gershman*	2026	37	**Related Theory**	●●●●○ 4/5	SELECTED	Fast weight programmers are explicitly unified under the NL paradigm in the original paper. Provides the neurobiological grounding NL draws from.	PDF
#26	Attention Is Not Retention: The Orthogonality Constraint in Infinite-Context Architectures	*Zahn, Beton, Chana*	2025	32	**Related Theory**	●●●●○ 4/5	SELECTED	Engages the attention-vs-retention distinction that NL resolves via its memory abstraction. Offers a theoretical counterpoint perspective.	PDF
#16	Prompt Injection Mitigation with Agentic AI, Nested Learning, and AI Sustainability via Semantic Caching	*Gosmar, Dah*	2026	33	**Application**	●●●○○ 3/5	SELECTED	Only paper besides the original with 'Nested Learning' in its title. Shows NL adoption in security and	PDF

#	Paper Title	Authors	Year	Pages	Category	Score	Status	Justification / Reason	PDF
								sustainability — adds domain diversity.	
#14	Vision Transformers That Never Stop Learning	*Sun, Yuan, Wang, Chen*	2026	26	**Application**	●●●○○ 3/5	**SELECTED**	Continual learning in ViTs corresponds directly to one of NL's stated goals. Provides a computer vision counterpoint to NLP-focused papers.	**PDF**
#13	AllMem: A Memory-centric Recipe for Efficient Long-context Modeling	*Wang, Wang, Peng, Qin, Kostelec, Sourmpis, Laborieux...*	2026	14	**Application**	●●●○○ 3/5	**SELECTED**	Memory-centric long-context approach aligns with NL's memory hierarchy. Short and dense — high information-per-chunk value.	**PDF**
#15	MSA: Memory Sparse Attention for Efficient End-to-End Memory Model Scaling to 100M Tokens	*Chen, Chen, Yi, Zhao, Li, Zhang, Sun, Hu, Han, Bing,...*	2026	20	**Application**	●●●○○ 3/5	**SELECTED**	Sparse attention for scalable memory addresses the computational efficiency question raised by NL. Concrete engineering contribution.	**PDF**
#8	Tiny Autoregressive Recursive Models	*Rauba, Fanconi, van der Schaar*	2026	15	**Related Theory**	●●●○○ 3/5	**SELECTED**	Recursive autoregressive structure mirrors NL's nested optimization formulation. Compact paper with clear theoretical contribution.	**PDF**
#22	Learning-to-Learn at Scale: Promises and Challenges	*Wang, Wang, Chen, Yu, Gan, Liu*	2026	7	**Related Theory**	●●●○○ 3/5	**SELECTED**	Meta-learning is the conceptual parent of NL's higher-order in-context learning. Dense survey enabling strong cross-paper comparisons.	**PDF**
SECTION B — EXCLUDED PAPERS (12 papers removed during curation)									
#1	Memory in the Age of AI Agents	*Hu, Liu, Yue, Zhang et al.*	2025	107	**Related Theory**	●●●○○ 3/5	**EXCLUDED**	107-page survey would dominate corpus. Mentions NL as one of many memory paradigms — lacks focused NL engagement. Excluded on size.	**PDF**
#9	ARO: A New Lens On Matrix Optimization For Large Models	*Gong, Zazo, Luo, Wang, Hensman, Ma*	2026	87	**Related Theory**	●●●○○ 3/5	**EXCLUDED**	Decent relevance but 87 pages would generate ~120 chunks from a single paper, disproportionately skewing corpus balance.	**PDF**
#2	Mechanistic Indicators of Understanding in Large Language Models	*Beckmann, Queloz*	2026	47	**Related Theory**	●●○○○ 2/5	**EXCLUDED**	Philosophical framing of LLM understanding. Likely a superficial NL citation. 47 pages of philosophical prose adds noise to technical queries.	**PDF**
#18	NOVAK: Unified Adaptive Optimizer for Deep Neural Networks	*Kavun*	2026	77	**Related Theory**	●●○○○ 2/5	**EXCLUDED**	Optimizer design is related to NL but at 77 pages with no confirmed deep NL engagement, the size-to-relevance trade-off is poor.	**PDF**
#7	A Mechanistic View on Video Generation as World Models: State and Dynamics	*Wang, Chen, Du, Yan, Ge, Shen, Xu, Wu, Chen, Xu, Ren...*	2026	17	**Application**	●●○○○ 2/5	**EXCLUDED**	Video generation world models are tangentially related to NL's state and memory concepts, but NL	**PDF**

#	Paper Title	Authors	Year	Pages	Category	Score	Status	Justification / Reason	PDF
								engagement is not confirmed at sufficient depth.	
#19	CFNN: Continued Fraction Neural Network	*Wang, Zhou, Hou, Cheng, Ding*	2026	40	**Weak Citation**	●●○○○ 2/5	**EXCLUDED**	Novel architecture using continued fractions. Architecturally unrelated to NL's multi-level optimization framework. Likely a superficial background citation.	**PDF**
#20	Spatial PDE-aware Selective State-space with Nested Memory for Mobile Traffic Forecasting	*Bettouche, Ali, Fischer, Kassler*	2026	3	**Weak Citation**	●●○○○ 2/5	**EXCLUDED**	Only 3 pages — insufficient content for meaningful chunking. Uses 'nested memory' in a narrow engineering context unrelated to NL theory.	**PDF**
#12	Learning to Predict, Discover, and Reason in High-Dimensional Event Sequences	*Faculty of Applied CS, Augsburg*	2026	201	**Weak Citation**	●○○○○ 1/5	**EXCLUDED**	201-page thesis would single-handedly exceed the rest of the corpus. Almost certainly cites NL only in passing. Impractical for a focused RAG corpus.	**PDF**
#21	An Axiomatic Approach to General Intelligence: SANC(E3)	*Kwon, Paeng*	2026	20	**Weak Citation**	●○○○○ 1/5	**EXCLUDED**	Philosophical general intelligence framework. Likely cites NL in a broad survey sweep with no engagement with NL's mathematical framework.	**PDF**
#23	Direct Optimal Action Learning	*Anonymous*	2025	23	**Weak Citation**	●○○○○ 1/5	**EXCLUDED**	Anonymous submission. Optimal action/RL framing is distant from NL's memory-optimizer paradigm. No confirmed NL relevance.	**PDF**
#24	The Emperor's New Algorithms: A Phenomenological Analysis of Synthetic Truth	*Alpay*	2025	21	**Weak Citation**	●○○○○ 1/5	**EXCLUDED**	Philosophical and sociological LLM critique. Zero technical relevance to NL's mathematical framework. Background citation only.	**PDF**
#25	Not All Who Wander Are Lost: Hallucinations as Neutral Dynamics in Residual Transformers	*Anonymous*	2025	26	**Weak Citation**	●○○○○ 1/5	**EXCLUDED**	Anonymous hallucination paper. Topic is completely orthogonal to NL's nested optimization paradigm. No substantive NL engagement.	**PDF**
#27	Mitigating Sycophancy and Hallucination in LLMs: The LOGOS Case Study	*Davalas, Angelaki*	2025	13	**Weak Citation**	●○○○○ 1/5	**EXCLUDED**	LLM alignment and hallucination paper. Unrelated to NL's theoretical contributions. Likely cites NL only in a broad related-work sweep.	**PDF**

Notes: Relevance score is on a 1–5 scale (5 = most relevant). Score dots: ●=filled, ○=empty. PDF links open the source document. All papers retrieved from publicly accessible repositories (arXiv, Springer, ResearchGate, TechRxiv, OpenReview).

The 15 selected papers split across three categories:
→Papers 17, 10, 11, 4, 5 are your **Core Extensions** — these have the deepest NL engagement. Papers 17 and 10 are both from Behrouz's own lab and explicitly built on the NL paradigm. Paper 11 even uses "nested objective" verbatim in the abstract.
→Papers 6, 16, 14, 13, 15 are your **Applications** — they show NL being adopted in different domains (anomaly detection, security, ViTs, long-context modeling). Good for questions like "how has NL been applied outside language modeling?"

→Papers 3, 26, 8, 22 are your **Related Theory** — fast weight programmers, attention-vs-retention, recursive models, and meta-learning at scale. These provide theoretical context and let you ask comparative questions like "how does NL relate to earlier memory-augmented architectures?"

The three biggest exclusions to note:

- Paper 1 (107 pages) and Paper 12 (201-page thesis) were size-kills — they'd dwarf everything else
- Papers 24, 25, 27 are genuinely unrelated and only cite NL for background survey padding
- Paper 9 (ARO, 87 pages) had decent relevance, but the page count was too disproportionate

The 16-paper set is balanced, dense, and will generate rich cross-paper comparison questions — exactly what your RAG system needs to demonstrate superiority over basic search.

2.4 Preprocessing (notebook used: Preprocessing.ipynb)

All 15 PDFs were then processed using PyMuPDF (fitz) to extract raw text. A cleaning pipeline was then applied to each extracted text file with the following steps:
(1) the reference section was identified using regex pattern matching on common headers such as "References", "REFERENCES", and "Bibliography", and everything following the first match was removed to eliminate citation noise.
(2) standalone page numbers appearing as isolated lines were stripped.
(3) runs of three or more consecutive blank lines were collapsed into two
(4) decorative divider lines consisting only of dashes or underscores were removed.

The cleaned text files were saved as UTF-8 encoded .txt files for downstream chunking.

FINAL CORPUS STATISTICS:

```
======================================================================
  CORPUS STATISTICS — 15 documents
======================================================================
  File                                          Words     Chars   Sentences
  ----------------------------------------------------------------------
  00_nested_learning_original.txt              26,626   168,020       2,001
  03_fast_weight_programming.txt               13,750    86,115         813
  04_end_to_end_ttt_long_context.txt           12,128    73,379         869
  05_dynamic_nested_hierarchies.txt             4,862    33,548         416
  06_crad_hope_anomaly_detection.txt            8,253    58,586       1,223
  08_tiny_autoregressive_recursive.txt          4,750    32,244         286
  10_hydra_dual_exponentiated_memory.txt        6,018    37,740         785
  11_beyond_ttt_optimal_control.txt             8,287    53,567         687
  13_allmem_long_context.txt                    4,971    35,316         442
  14_vision_transformers_never_stop.txt         4,941    33,622         360
  15_msa_memory_sparse_attention.txt            7,680    54,715         786
  16_prompt_injection_nested_learning.txt      11,109    79,657         785
  17_memory_caching_rnns.txt                    8,389    49,517       1,484
  22_learning_to_learn_at_scale.txt             3,434    24,342         185
  26_attention_not_retention.txt               14,811   102,921       1,010

======================================================================
  TOTALS
======================================================================
  Documents         : 15
  Total words       : 140,009
  Total characters  : 923,289
  Total sentences   : 12,132
  Avg words/doc     : 9,333
  Min words/doc     : 3,434  (22_learning_to_learn_at_scale.txt)
```

3. Algorithms, Models, and Retrieval Methods:

3.1 CHUNKING STRATEGIES:

Text chunking is the process of segmenting cleaned documents into smaller units suitable for embedding and retrieval. Chunk size directly affects retrieval quality chunks that are too large carry mixed topics that dilute semantic signal, while chunks that are too small lose the contextual information needed for coherent answer generation.
To systematically evaluate the impact of chunking on system performance, three distinct strategies were implemented and compared.

Strategy A — Fixed Character Splitting (Baseline) The first strategy used LangChain's CharacterTextSplitter with a chunk size of 512 characters and an overlap of 50 characters, splitting only on newline boundaries. This approach makes no attempt to respect sentence or paragraph structure and serves purely as a baseline for comparison. It is the simplest and fastest strategy but frequently cuts mid-sentence or mid-theorem, which is particularly problematic for academic text containing dense theoretical notation. **This strategy produced 2,007 chunks** across the 15 documents with an average chunk size of 497 characters.	**Strategy B — Recursive Character Splitting (Production)** The second strategy used LangChain's RecursiveCharacterTextSplitter with a chunk size of 400 characters and an overlap of 80 characters. Rather than splitting at a fixed character boundary, this splitter attempts to find the most natural nearby break point by trying separators in order of preference: paragraph breaks (\n\n), line breaks (\n), sentence boundaries (.), and word boundaries ( ). This preserves semantic coherence by ensuring that chunks respect the document's own structure wherever possible. The larger overlap of 80 characters (compared to 50 in Strategy A) ensures that key sentences near chunk boundaries appear in both adjacent chunks, reducing the risk of relevant content being missed during retrieval. **This strategy produced 2,753 chunks** with an average chunk size of 336 characters and was selected as the production strategy based on its superior balance of coherence and coverage. It was also selected as the production strategy for the deployed application based on its performance in the ablation study described in Section D.	**Strategy C — Semantic Chunking** The third strategy used LangChain's SemanticChunker powered by the all-MiniLM-L6-v2 embedding model. Unlike the character-based strategies, this approach embeds every sentence individually and computes cosine similarity between adjacent sentence pairs. A split is introduced wherever the similarity drops below the 70th percentile threshold, meaning the text is only divided when the topic genuinely changes. This is the most semantically intelligent of the three strategies but also the most computationally expensive, as it requires running the embedding model during the chunking phase itself rather than only at indexing time. Any resulting chunks shorter than 80 characters were discarded as micro-chunks caused by abbreviation mis-splits common in academic text (e.g. "et al.", "Fig.", "Eq."). **This strategy produced 1,455 chunks** with larger average chunk size reflecting its tendency to merge topically consistent sentences into longer units. Semantic chunking was executed with GPU acceleration on Kaggle (T4 GPU) to reduce processing time.

STRATEGY	CHUNKS	AVG SIZE	TIME
A — Fixed	2,007	467c	0.1s
B- Recursive	2753	354c	0.1s
C-Semantic	1455	625c	10.4s

3.2 RETRIEVAL METHODS:

The RAG pipeline combines four retrieval components working in sequence: BM25 keyword search, semantic vector search via Pinecone, Reciprocal Rank Fusion (RRF) to merge results, and Cross-Encoder reranking to produce the final set of retrieved chunks. This multi-stage approach was chosen because no single retrieval method adequately handles all query types across dense academic text. The following subsections describe each component.

1. **BM25 — Keyword Search**

BM25 (Best Match 25) is a probabilistic ranking function that scores documents based on term frequency, inverse document frequency, and document length normalisation. It is particularly effective for retrieving chunks containing specific technical terminology — model names, mathematical concepts, paper-specific notation — where semantic search may fail due to vocabulary mismatch between the query and the stored chunks. For example, a query containing "HYDRA" or "Memory Caching" will score those exact terms highly in BM25 even if the semantic embedding of the query does not perfectly align with the chunk embedding.

A separate BM25 index was built for each of the three chunking strategies to ensure a fair and clean ablation comparison. Each BM25Okapi object (from the rank-bm25 library) was constructed by tokenising every chunk in the corresponding corpus pkl file using lowercase conversion and whitespace splitting. This means Strategy A's hybrid retrieval uses BM25 built from its own 2,007 chunks, Strategy B uses BM25 from its 2,753 chunks, and Strategy C uses BM25 from its 1,455 chunks. The indexes are held in memory during retrieval and rebuilt at the start of each evaluation session. At query time, the user's query is tokenised identically and passed to bm25.get_scores(), which returns a relevance score for every chunk in the corpus. The top 15 highest-scoring chunks are taken as the BM25 candidate set.

2. **Semantic Search — Pinecone**

For each query, the query text is first embedded using text-embedding-3-small to produce a 1,536-dimensional vector. This vector is sent to the corresponding Pinecone index (selected based on which chunking strategy is being evaluated) via the index.query() call with top_k=15. Pinecone returns the 15 chunks whose stored vectors have the highest cosine similarity to the query vector. These 15 results form the semantic candidate set.

3. **Reciprocal Rank Fusion (RRF)**

The BM25 candidate set (15 chunks, ranked by keyword score) and the semantic candidate set (15 chunks, ranked by cosine similarity) are merged using Reciprocal Rank Fusion. RRF was chosen over score-based fusion because BM25 scores and cosine similarity scores operate on completely incompatible scales — BM25 produces unbounded positive values while cosine similarity is bounded between -1 and 1. Direct addition or weighted averaging of these scores would be meaningless. RRF avoids this problem entirely by using only the rank position of each chunk rather than its raw score.

The RRF formula assigns a score to each unique chunk as follows:

RRF score = $\Sigma\ 1 / (k + \text{rank})$

where k = 60 (the standard constant that dampens the influence of very high-ranked documents) and rank is the position of the chunk in each list it appears in, starting from 1. If a chunk appears in both the BM25 list and the semantic list, its contributions from both are summed — chunks relevant to both keyword matching and semantic similarity receive a double boost. All unique chunks from both lists are then ranked by their combined RRF score, and the top 15 are taken forward as the merged candidate set for reranking.

4. **Cross-Encoder Reranking**

The final stage passes the top 15 RRF-merged candidates through a Cross-Encoder model (cross-encoder/ms-marco-MiniLM-L-6-v2). Unlike the bi-encoder approach used for initial retrieval — where the query and each chunk are embedded independently and compared — a Cross-Encoder processes the query and each candidate chunk together as a single concatenated input. This joint encoding allows the model to directly model interactions between query tokens and chunk tokens, producing a much more accurate relevance score than embedding similarity alone.

The Cross-Encoder produces a scalar relevance score for each of the 15 (query, chunk) pairs. These 15 pairs are then sorted in descending order by score and the top 5 chunks are selected as the final retrieved context for answer generation. This two-stage retrieval design — fast approximate retrieval followed by accurate reranking — is a standard pattern in production RAG systems and is a key contributor to the system's retrieval quality as demonstrated in the ablation results in Section D.

3.3 ALL MODELS USED THROUGHOUT THE SYSTEM.

Model	Provider	Role
text-embedding-3-small	OpenAI	Document & query embedding; relevancy cosine similarity

Model	Provider	Role
cross-encoder/ms-marco-MiniLM-L-6-v2	HuggingFace	Cross-encoder re-ranking
gpt-4o-mini	OpenAI	Answer generation (primary); LLM-as-a-Judge (primary)
llama-3.1-8b-instant	Groq	Answer generation (comparison); LLM-as-a-Judge (alternative)
llama-3.3-70b-versatile	Groq	Golden set generation
BM25Okapi (rank-bm25)	Open-source	Keyword retrieval

1. Embedding Model — text-embedding-3-small (OpenAI)
Role: Document embedding (offline, at indexing time) and query embedding (online, at retrieval time). Also used during relevancy evaluation to compute cosine similarity between the original query and the three LLM-generated surrogate questions.

All document chunks across the three chunking strategies (Fixed, Recursive, Semantic) were embedded using OpenAI's text-embedding-3-small, producing 1,536-dimensional vectors that were upserted into their respective Pinecone indexes. The same model encodes every incoming user query at retrieval time before the Pinecone nearest-neighbour search.

Why we chose text-embedding-3-small over all-MiniLM-L6-v2?

The primary reason is retrieval accuracy. Our corpus consists of dense academic papers on machine learning — content with highly specific terminology, nuanced technical distinctions, and domain-specific phrasing. The 1,536-dimensional embedding space of text-embedding-3-small provides significantly richer representational capacity than the 384-dimensional space of all-MiniLM-L6-v2, allowing semantically similar but lexically different passages to be more reliably matched. This is reflected in its higher MTEB benchmark score.

A second practical reason is consistency. Since our document embeddings were pre-computed and stored in Pinecone using text-embedding-3-small, the query encoder must be the same model — embedding spaces are model-specific, and mixing models would make cosine similarity scores meaningless. Using all-MiniLM-L6-v2 for query encoding against a text-embedding-3-small index would produce entirely incorrect retrieval results.

all-MiniLM-L6-v2 would have been a reasonable choice in a resource-constrained deployment where the embedding model must run locally inside the Hugging Face Space with no API budget. In our setup, however, the marginal API cost was acceptable given the improvement in retrieval quality, and the ablation results in Table X confirm that the semantic retrieval stage contributes meaningfully to the final Faithfulness and Relevancy scores.

2. Cross-Encoder Re-ranker — cross-encoder/ms-marco-MiniLM-L-6-v2 (Sentence Transformers)
Role: Re-ranking the top-15 RRF-merged candidates to produce the final top-5 chunks passed to the generator.

Unlike the bi-encoder approach used in semantic search — where the query and each chunk are encoded independently — a cross-encoder concatenates the query and chunk into a single input and produces a joint relevance score. This allows the model to directly attend to token-level interactions between the query and the candidate chunk, yielding a much more precise relevance judgement than cosine similarity alone.

ms-marco-MiniLM-L-6-v2 was selected specifically because it was fine-tuned on the MS MARCO passage retrieval dataset, which consists of real user queries paired with relevant passages — a setting closely analogous to our RAG use case. The MiniLM-L-6 variant was chosen over larger cross-encoders (e.g., L-12) to balance re-ranking accuracy against inference latency, as the cross-encoder is applied to 15 candidates on every query.

3. Answer Generation Model (Primary) — gpt-4o-mini (OpenAI)
Role: Generating answers from the retrieved context across all six ablation configurations in the main ablation run.

gpt-4o-mini served as the primary answer generator. It was given a strict system prompt instructing it to answer only from the retrieved context, explicitly prohibiting fabrication.

Why gpt-4o-mini was chosen as the primary model:

gpt-4o-mini is a commercially fine-tuned model with strong instruction-following behaviour, particularly on the kind of strict prompt we used — answering only from retrieved context and explicitly refusing to fabricate. On dense academic content, it consistently produced well-structured, precise answers that mapped cleanly onto the retrieved chunks, which directly benefits the Faithfulness score. The API was purchased specifically for this project because retrieval quality alone does not guarantee answer quality; the generator must be capable of faithfully synthesising the retrieved context without hallucinating unsupported claims.

4. Answer Generation Model (Comparison) — llama-3.1-8b-instant (Groq)
Role: Alternative generator in the secondary ablation run, enabling a direct comparison of answer quality between a closed-source and an open-source model under identical retrieval conditions.
The same six ablation configurations were re-run with llama-3.1-8b-instant as the generator, keeping retrieval, re-ranking, and evaluation identical. This isolates the effect of the generation model on Faithfulness and Relevancy scores. Groq's inference API was used because it delivers extremely low generation latency for open-source models, making it practical to run across 180 evaluations (6 configs × 30 questions) without excessive wall-clock time. The llama-3.1-8b-instant model was chosen as a compact, fast open-source alternative to allow a fair cost- and speed-adjusted comparison against gpt-4o-mini.

Why llama-3.1-8b-instant was included:

Rather than testing only the paid model, we included llama-3.1-8b-instant as a free baseline to understand the real cost–quality trade-off. Groq's inference infrastructure delivers very low generation latency for this model, making it fast and completely free to use. At 8 billion parameters it is considerably smaller than gpt-4o-mini, but it is a modern, instruction-tuned model capable of following our RAG prompt structure. Including it allowed us to answer a practical question: how much quality are you actually giving up by using a free model instead of a paid one?

5. LLM-as-a-Judge (Primary) — gpt-4o-mini (OpenAI)
Role: Automated faithfulness evaluation — extracting factual claims from generated answers and verifying each claim against the retrieved context. For Faithfulness scoring, gpt-4o-mini was used as the judge in two sequential prompts: the first instructs it to extract all distinct factual claims from the generated answer as a numbered list; the second verifies each claim individually against the retrieved context, returning a binary YES/NO verdict. The Faithfulness score is then computed as the proportion of verified claims. gpt-4o-mini was chosen as the judge for the same reasons as generation — reliable instruction-following and low latency — and because using a strong model as judge is standard practice in LLM-as-a-Judge pipelines to reduce false positives and negatives in claim verification.

6. LLM-as-a-Judge (Alternative) — llama-3.1-8b-instant (Groq)
Role: Alternative judge in Cell 19 of the ablation notebook, used to assess whether judge-model choice meaningfully affects the reported evaluation scores.
To validate that our Faithfulness and Relevancy scores are not an artefact of using gpt-4o-mini as both generator and judge — a known source of self-evaluation bias — we re-evaluated the best configuration (Config 5: Recursive Chunking + Hybrid + Re-ranking) using llama-3.1-8b-instant as the judge while keeping gpt-4o-mini as the generator. Comparing scores across both judge configurations allowed us to assess the robustness of our evaluation pipeline to judge-model choice.

7. Golden Set Generation Model — llama-3.3-70b-versatile (Groq)
Role: Generating the 30-question golden evaluation set used across all ablation configurations.
The golden set of 30 technical question–answer pairs was generated using llama-3.3-70b-versatile via the Groq API. For each paper in the corpus, the first four chunks (abstract and introduction — the densest, most factually rich sections) were concatenated and passed to the model with a strict prompt requiring: questions answerable only from the provided text, reasoning-based rather than surface-level questions, and responses in valid JSON. Temperature was set to 0.2 to ensure diverse but grounded output.
llama-3.3-70b-versatile was selected for this task — rather than gpt-4o-mini — because its larger parameter count produces more nuanced and technically demanding questions on academic content, which is desirable for a golden set intended to stress-test the retrieval pipeline. Using a different model for golden set generation than for answer generation also reduces the risk of model-specific bias inflating evaluation scores.

3.4 Prompt Structure:

Our system uses five distinct prompt types across three stages: answer generation, faithfulness evaluation, and relevancy evaluation. For each stage we also developed an improved prompt variant as part of a dedicated prompt engineering ablation (Configurations P1–P4). The golden set generation prompt is described separately at the end.

1. Answer Generation Prompt (Baseline — P1)
Used by both gpt-4o-mini and llama-3.1-8b-instant across all six main ablation configurations. Temperature was set to 0.1 to keep outputs deterministic and grounded.

`You are a research assistant specializing in machine learning.`
`Answer using ONLY the provided context from academic papers.`
`Do not fabricate facts or citations.`
`If context is insufficient, say so explicitly.`
 	
`Context:`
`[Source: <paper title> (<year>)]`
`<chunk text>`
 	
`---`
 	
`[Source: <paper title> (<year>)]`
`<chunk text>`
 	
`Question: {query}`
`Answer:`
	Design rationale: The prompt establishes a strict persona and a hard constraint against fabrication. The retrieved chunks are formatted with source labels and separated by horizontal rules to clearly delineate document boundaries. The low temperature (0.1) reduces hallucination risk. The explicit instruction to acknowledge insufficient context prevents the model from generating plausible-sounding but unsupported answers when retrieval fails.

2. Answer Generation Prompt (Improved — P2 / P4)

Used in the prompt engineering ablation. Introduces system/user role separation, chain-of-thought reasoning, mandatory inline citations, and structured output.

```
System message:

You are an expert research assistant who answers questions strictly from
provided academic-paper excerpts. You NEVER introduce facts, numbers, or
claims not present in the excerpts. When you use a fact, cite its source
inline as (Source N). If the excerpts do not contain enough information,
state that clearly.


User message:

Below are numbered excerpts from academic papers. Read them carefully.

[Source 1: <paper title> (<year>)]
<chunk text>

---

[Source 2: <paper title> (<year>)]
<chunk text>

---

QUESTION: {query}

Think step by step:
1. Identify which sources are relevant to the question.
2. Extract the key facts from those sources.
3. Synthesise a concise, well-cited answer.

Respond in this exact format:

[Reasoning]
<2-4 sentences identifying the relevant sources and their key points>

[Answer]
<Your final answer with inline citations like (Source 1), (Source 2), etc.>
```

Design rationale: Separating the system and user roles gives the model clearer behavioural boundaries. The numbered source format enables inline citations, making it possible to trace every claim back to a specific chunk. The explicit chain-of-thought step encourages the model to reason before committing to an answer, reducing the chance of latching onto the first relevant sentence. Only the [Answer] block is passed downstream to the evaluator.

3. Faithfulness Judge Prompt — Step 1: Claim Extraction (Baseline)

`Extract all distinct factual claims from this answer.`
`Return ONLY a numbered list, one claim per line. No preamble or explanation.`
 	
`Answer: {answer}`
	Design rationale: Keeping this prompt minimal forces the model to output a clean numbered list with no surrounding commentary, which simplifies parsing. Claims shorter than 15 characters are discarded programmatically to filter out trivial outputs.

4.Faithfulness Judge Prompt — Step 2: Claim Verification (Baseline)
Applied individually for each extracted claim. Temperature is set to 0.0 for deterministic binary verdicts.

`Context from academic papers:`
`{context[:2500]}`
 	
`Is this claim directly supported by the context above?`
`Answer with YES or NO only. No explanation.`
 	
`Claim: {claim}`
	Design rationale: Each claim is verified in isolation rather than all at once, preventing the model from conflating evidence across claims. The binary YES/NO format eliminates ambiguity in parsing the verdict. Context is capped at 2,500 characters to stay within the token budget while covering the most relevant retrieved content.

5. Faithfulness Judge Prompt — Step 1: Claim Extraction (Improved)
Used in prompt configurations P3 and P4. Introduces explicit atomic decomposition rules.

`You are evaluating a RAG system answer for factual faithfulness.`
 	
`Task: Extract every distinct, atomic factual claim from the answer below.`
`Rules:`
`- One claim per line, numbered.`
`- Each claim must be a single verifiable statement (not compound).`
`- Include numerical values, comparisons, and named entities as separate claims.`
`- Exclude vague statements like "the paper discusses..." - only concrete facts.`
`- No preamble, no explanation - just the numbered list.`
 	
`Answer to decompose:`
`{answer}`
	Design rationale: The baseline extraction prompt occasionally produced compound claims (e.g., "X uses method Y and achieves Z%") which are difficult to verify atomically. The improved version explicitly prohibits compound statements and calls out common failure modes such as vague meta-commentary, resulting in cleaner, more verifiable claim lists.

6. Faithfulness Judge Prompt — Step 2: Claim Verification (Improved)
Replaces binary YES/NO with a three-level rubric: SUPPORTED, PARTIAL, or NOT_SUPPORTED, mapped to credit scores of 1.0, 0.5, and 0.0 respectively. Context window is also extended to 3,500 characters.

`You are a strict factual verifier for academic RAG systems.`
 	
`Context (excerpts from academic papers):`
`{context[:3500]}`
 	
`Claim to verify: "{claim}"`
 	
`Evaluate whether the claim is supported by the context above.`
`Use EXACTLY one of these three verdicts:`
`  SUPPORTED  - the context explicitly states or clearly implies this claim`
`  PARTIAL  - the context partially supports it but key details are missing`
`  NOT_SUPPORTED - the context does not support this claim or contradicts it`
 	
`Respond with a single word: SUPPORTED, PARTIAL, or NOT_SUPPORTED`
	Design rationale: Binary YES/NO scoring treats a partially supported claim identically to a completely unsupported one, which overly penalises answers that are mostly grounded but miss a specific detail. The three-level rubric produces a more honest and granular faithfulness score. This stricter judge typically produces lower scores than the baseline, which is by design — the improved score is a more accurate reflection of true faithfulness.

7. Relevancy Judge Prompt (Baseline)

`Generate exactly 3 questions that this answer is responding to.`
`Return ONLY the 3 questions, one per line. No numbering, no explanation.`
 	
`Answer: {answer}`
	The three generated questions are then embedded using text-embedding-3-small and their cosine similarities with the original query embedding are averaged to produce the Relevancy score.
	Design rationale: Rather than directly scoring relevancy — which is difficult to prompt reliably — this approach uses question generation as a proxy. If the generated answer is relevant to the original query, the questions that the answer implies should embed close to the original query in vector space. This method is adapted from the RAGAS framework's answer relevancy metric.

8. Relevancy Judge Prompt (Improved)

`You are evaluating how well a RAG system answer addresses a user question.`
 	
`QUESTION: {query}`
 	
`ANSWER: {answer}`
 	
`Score the answer on the following 4 aspects. For each, give a score from`
`0.0 to 1.0 (step 0.25) and a one-sentence justification.`
 	
`Aspects:`
`1. topic_alignment  - Does the answer address the same topic as the question?`
`2. specificity_match  - Does the answer provide the level of detail asked for?`
`3. scope_match  - Does the answer stay within scope (no irrelevant tangents)?`
`4. completeness  - Does the answer address all parts of the question?`
 	
`Respond ONLY with valid JSON (no markdown, no code fences):`
`{"topic_alignment":  {"score": <0-1>, "reason": "<sentence>"},`
`  "specificity_match": {"score": <0-1>, "reason": "<sentence>"},`
`  "scope_match":  {"score": <0-1>, "reason": "<sentence>"},`
`  "completeness":  {"score": <0-1>, "reason": "<sentence>"}}`
	The four aspect scores are combined using a weighted mean: topic_alignment (0.35), specificity_match (0.25), scope_match (0.20), completeness (0.20). If JSON parsing fails, the method falls back to the baseline embedding similarity approach.
	Design rationale: The baseline question-generation proxy can score high even when the answer is only tangentially related to the query, because generated questions tend to mimic the answer's own vocabulary. The aspect-decomposed rubric evaluates relevancy directly and multi-dimensionally, with topic alignment weighted highest since it captures the most fundamental relevancy criterion.

9. Golden Set Generation Prompt:

`You are an expert AI researcher specializing in machine learning.`
 	
`Based on the following excerpt from an academic paper, generate exactly 2 high-quality technical questions with detailed answers.`
 	
`STRICT RULES:`
`- Questions must be answerable ONLY from the provided text`
`- Questions must require reasoning and understanding, not just copying a sentence`
`- Answers must be detailed, specific, and grounded in the text`
`- Do NOT ask trivial questions like "what is the paper about"`
`- Focus on concepts like: how methods work, why design choices were made, what limitations exist, how this relates to Nested Learning`
`- Return ONLY valid JSON, no markdown, no backticks, no extra text`
`- Format exactly: [{"question": "...", "answer": "..."}]`
 	
`Paper Title: {title}`
 	
`Excerpt:`
`{sample}`
	Design rationale: The prompt explicitly prohibits surface-level questions and steers the model toward questions requiring comprehension and reasoning rather than simple text retrieval. The JSON-only output format ensures clean, programmatically parseable responses. Using the first four chunks per paper (abstract and introduction) as the excerpt maximises the density of key concepts per token. Q&A pairs shorter than 20 characters (question) or 50 characters (answer) were discarded programmatically to filter degenerate outputs.

3.5 ALL CONFIGURATIONS TESTED

The table below documents all 19 configurations evaluated across five experiment groups. Each row represents a unique combination of chunking strategy, retrieval mode, generator model, temperature, and prompt variant. Configurations sharing the same row fill colour belong to the same experiment group.

Config ID	Experiment Group	Chunking Strategy	Retrieval Mode	Generator Model	Temp.	Generation Prompt	Faithfulness Judge	Faithfulness Prompt	Relevancy Judge	Relevancy Prompt
C1	Main Ablation	Fixed (A)	Semantic Only	gpt-4o-mini	0.1	Baseline	gpt-4o-mini	Binary YES/NO	gpt-4o-mini	Q-Gen + Cosine
C2	Main Ablation	Recursive (B)	Semantic Only	gpt-4o-mini	0.1	Baseline	gpt-4o-mini	Binary YES/NO	gpt-4o-mini	Q-Gen + Cosine
C3	Main Ablation	Semantic (C)	Semantic Only	gpt-4o-mini	0.1	Baseline	gpt-4o-mini	Binary YES/NO	gpt-4o-mini	Q-Gen + Cosine
C4	Main Ablation	Fixed (A)	Hybrid + Rerank	gpt-4o-mini	0.1	Baseline	gpt-4o-mini	Binary YES/NO	gpt-4o-mini	Q-Gen + Cosine
C5 ★	**Main Ablation**	**Recursive (B)**	**Hybrid + Rerank**	**gpt-4o-mini**	**0.1**	**Baseline**	**gpt-4o-mini**	**Binary YES/NO**	**gpt-4o-mini**	**Q-Gen + Cosine**
C6	Main Ablation	Semantic (C)	Hybrid + Rerank	gpt-4o-mini	0.1	Baseline	gpt-4o-mini	Binary YES/NO	gpt-4o-mini	Q-Gen + Cosine
C7	Generator Comparison	Fixed (A)	Semantic Only	llama-3.1-8b-instant	0.1	Baseline	gpt-4o-mini	Binary YES/NO	gpt-4o-mini	Q-Gen + Cosine
C8	Generator Comparison	Recursive (B)	Semantic Only	llama-3.1-8b-instant	0.1	Baseline	gpt-4o-mini	Binary YES/NO	gpt-4o-mini	Q-Gen + Cosine
C9	Generator Comparison	Semantic (C)	Semantic Only	llama-3.1-8b-instant	0.1	Baseline	gpt-4o-mini	Binary YES/NO	gpt-4o-mini	Q-Gen + Cosine

Config ID	Experiment Group	Chunking Strategy	Retrieval Mode	Generator Model	Temp.	Generation Prompt	Faithfulness Judge	Faithfulness Prompt	Relevancy Judge	Relevancy Prompt
C10	Generator Comparison	Fixed (A)	Hybrid + Rerank	llama-3.1-8b-instant	0.1	Baseline	gpt-4o-mini	Binary YES/NO	gpt-4o-mini	Q-Gen + Cosine
C11	Generator Comparison	Recursive (B)	Hybrid + Rerank	llama-3.1-8b-instant	0.1	Baseline	gpt-4o-mini	Binary YES/NO	gpt-4o-mini	Q-Gen + Cosine
C12	Generator Comparison	Semantic (C)	Hybrid + Rerank	llama-3.1-8b-instant	0.1	Baseline	gpt-4o-mini	Binary YES/NO	gpt-4o-mini	Q-Gen + Cosine
C13	Judge Comparison	Recursive (B)	Hybrid + Rerank	gpt-4o-mini	0.1	Baseline	llama-3.1-8b-instant	Binary YES/NO	llama-3.1-8b-instant	Q-Gen + Cosine
C14	Temp. Ablation	Recursive (B)	Hybrid + Rerank	gpt-4o-mini	0.7	Baseline	gpt-4o-mini	Binary YES/NO	gpt-4o-mini	Q-Gen + Cosine
C15	Temp. Ablation	Recursive (B)	Hybrid + Rerank	llama-3.1-8b-instant	0.7	Baseline	gpt-4o-mini	Binary YES/NO	gpt-4o-mini	Q-Gen + Cosine
P1	Prompt Ablation	Recursive (B)	Hybrid + Rerank	gpt-4o-mini	0.1	Baseline	gpt-4o-mini	Binary YES/NO	gpt-4o-mini	Q-Gen + Cosine
P2	Prompt Ablation	Recursive (B)	Hybrid + Rerank	gpt-4o-mini	0.1	Improved (CoT+Cite)	gpt-4o-mini	Binary YES/NO	gpt-4o-mini	Q-Gen + Cosine
P3	Prompt Ablation	Recursive (B)	Hybrid + Rerank	gpt-4o-mini	0.1	Baseline	gpt-4o-mini	3-Level Rubric	gpt-4o-mini	Aspect-Decomposed
P4	Prompt Ablation	Recursive (B)	Hybrid + Rerank	gpt-4o-mini	0.1	Improved (CoT+Cite)	gpt-4o-mini	3-Level Rubric	gpt-4o-mini	Aspect-Decomposed

Legend: C5 ★ = Best overall configuration (Recursive Chunking + Hybrid + Rerank + gpt-4o-mini). P1 reuses C5 results as the prompt ablation baseline. Improved (CoT+Cite) = Chain-of-thought generation with mandatory inline citations. 3-Level Rubric = SUPPORTED / PARTIAL / NOT_SUPPORTED faithfulness scoring. Aspect-Decomposed = 4-aspect weighted relevancy scoring (topic alignment, specificity, scope, completeness).

4. Performance Metrics

Section D presents a comprehensive evaluation of all configurations tested across five experiment groups. Evaluation metrics cover faithfulness and relevancy (answer quality), inference latency (retrieval time, generation time, total response time), and cross-dimensional analyses comparing chunking strategies, retrieval modes, generator models, temperature settings, and prompt variants. All quality scores were computed using the custom LLM-as-a-Judge pipeline described in Section C.

4.1 Master Results Table - All Configurations

Table D.1 consolidates every measured configuration. Faithfulness reflects the proportion of answer claims verifiable against retrieved context; Relevancy reflects semantic alignment between generated answer and original query. Latency columns are in milliseconds. C5 is the best main ablation configuration (shaded blue). P3/P4 rows use the improved judge prompt which recalibrates the scoring scale and are not directly comparable to C1-C15 on raw numbers (shaded green).

Config	Chunking	Retrieval	Generator	Faith.	Relev.	Ret.(ms)	Gen.(ms)	Total(ms)
C1	Fixed (A)	Semantic Only	gpt-4o-mini	0.925	0.688	352	2,721	3,073
C2	Recursive (B)	Semantic Only	gpt-4o-mini	0.964	0.696	270	4,020	4,290
C3	Semantic (C)	Semantic Only	gpt-4o-mini	0.836	0.703	267	4,243	4,509
C4	Fixed (A)	Hybrid+Rerank	gpt-4o-mini	0.966	0.714	239	2,703	2,942
C5 (Best)	Recursive (B)	Hybrid+Rerank	gpt-4o-mini	0.982	0.709	229	2,832	3,061
C6	Semantic (C)	Hybrid+Rerank	gpt-4o-mini	0.832	0.692	239	3,857	4,096
C7	Fixed (A)	Semantic Only	llama-3.1-8b	0.944	0.703	242	493	734
C8	Recursive (B)	Semantic Only	llama-3.1-8b	0.941	0.702	240	403	642
C9	Semantic (C)	Semantic Only	llama-3.1-8b	0.744	0.696	240	604	844
C10	Fixed (A)	Hybrid+Rerank	llama-3.1-8b	0.965	0.712	227	420	647
C11	Recursive (B)	Hybrid+Rerank	llama-3.1-8b	0.970	0.717	226	393	619
C12	Semantic (C)	Hybrid+Rerank	llama-3.1-8b	0.740	0.663	229	2,619	2,849
P1 (=C5)	Recursive (B)	Hybrid+Rerank	gpt-4o-mini / Baseline Judge	0.968	0.695	229	2,832	3,061
P2	Recursive (B)	Hybrid+Rerank	gpt-4o-mini / Imp. Gen	1.000	0.749	229	3,459	3,688
P3	Recursive (B)	Hybrid+Rerank	gpt-4o-mini / Imp. Judge	0.990	0.925	229	2,958	3,187
P4	Recursive (B)	Hybrid+Rerank	gpt-4o-mini / Both Imp.	0.997	0.902	229	4,120	4,349

Note: C13 (Llama-as-Judge), C14 (GPT T=0.7), C15 (Llama T=0.7) were also evaluated but their primary purpose was confirming judge robustness and temperature sensitivity rather than head-to-head metric comparison; findings are discussed in the relevant subsections.

4.2 Chunking Strategy Comparison

Three chunking strategies were evaluated under both retrieval modes with gpt-4o-mini as generator. The results isolate the effect of chunking on faithfulness, relevancy, and latency.

4.2.1 Semantic-Only Retrieval (C1 vs C2 vs C3)

Strategy	Config	Corpus Chunks	Avg Chunk Size	Faithfulness	Relevancy	Ret.(ms)	Total(ms)
Fixed (A) - Baseline	C1	2,007	497 chars	0.925	0.688	352	3,073
Recursive (B) - Production	C2	2,753	336 chars	0.964 (+4.2%)	0.696 (+1.2%)	270	4,290
Semantic (C) - Intelligent	C3	1,455	~600 chars	0.836 (-9.6%)	0.703 (+2.2%)	267	4,509

Recursive chunking (C2) outperforms fixed chunking (C1) by 4.2 percentage points on faithfulness under semantic-only retrieval. This is attributable to Recursive chunking's structure-aware splitting: it respects paragraph and sentence boundaries, producing chunks that align more cleanly with the document's natural information units. When a chunk corresponds to a complete paragraph discussing a single concept, the retriever can match it precisely to relevant queries, and the generator's claims then map cleanly back onto the retrieved context.

Semantic chunking (C3) produces the lowest faithfulness score (0.836) despite the most sophisticated splitting logic. This is a counter-intuitive finding. The likely explanation is chunk size heterogeneity: semantic chunking produces a bimodal distribution of chunk sizes. Very long chunks frequently contain mixed sub-topics that produce diluted embedding vectors, making it harder for the retriever to identify the correct passage. When the wrong chunk is retrieved, the generator may produce a fluent answer from a less relevant context, increasing the chance of unsupported claims. Notably, Semantic chunking achieves the highest relevancy score (0.703) among the three, suggesting that when retrieval succeeds, the richer context aids topic alignment.

4.2.2 Hybrid + Rerank Retrieval (C4 vs C5 vs C6)

Strategy	Config	Faithfulness	Relevancy	Ret.(ms)	Gen.(ms)	Total(ms)
Fixed (A)	C4	0.966	0.714	239	2,703	2,942
Recursive (B) - Best	C5	0.982	0.709	229	2,832	3,061

Semantic (C)	C6	0.832	0.692	239	3,857	4,096

Under hybrid+rerank retrieval, the performance ordering is preserved: Recursive (B) > Fixed (A) > Semantic (C) on faithfulness. C5 achieves the highest faithfulness of any configuration in the main ablation (0.982). The addition of BM25 + reranking closes the gap between Fixed and Recursive: Fixed improves from 0.925 to 0.966 (+4.4pp), while Recursive improves from 0.964 to 0.982 (+1.8pp). Recursive remains superior because its structure-preserving chunks are more compatible with cross-encoder reranking - the cross-encoder's joint query-chunk encoding benefits from coherent information units.

Semantic chunking (C6) under hybrid retrieval shows the highest generation time (3,857ms vs 2,703-2,832ms for A and B). This is attributed to larger average chunk sizes requiring longer generation context. The faithfulness variance for C6 is also the highest among all configurations (per-question scores ranging from 0.125 to 1.0), indicating that semantic chunking produces inconsistent retrieval quality - it excels on some question types and fails badly on others, which is a critical production concern.

4.3 Retrieval Mode: Semantic-Only vs. Hybrid + Rerank

Pair	Chunking	Faith. Change	Rel. Change	Ret. Change	Total Change
C1->C4	Fixed (A)	0.925->0.966 (+0.041)	0.688->0.714 (+0.026)	352->239ms (-32%)	3,073->2,942ms (-131ms)
C2->C5	Recursive (B)	0.964->0.982 (+0.018)	0.696->0.709 (+0.013)	270->229ms (-15%)	4,290->3,061ms (-1,229ms)
C3->C6	Semantic (C)	0.836->0.832 (-0.004)	0.703->0.692 (-0.011)	267->239ms (-10%)	4,509->4,096ms (-413ms)

Hybrid + Rerank retrieval consistently outperforms semantic-only retrieval for Fixed and Recursive chunking on both metrics. The improvements are most pronounced for Fixed chunking (+4.1pp faithfulness, +2.6pp relevancy), confirming that BM25's exact-term matching compensates for Fixed chunking's weaker semantic coherence. When a chunk is split mid-sentence, keyword overlap with the query still allows BM25 to surface it, and the cross-encoder then filters out noise from both retrieval pools.

Semantic chunking (C6) shows marginal degradation under hybrid retrieval (-0.4pp faithfulness, -1.1pp relevancy). This is a corner case arising from an interaction between semantic chunking's larger chunks and BM25 retrieval: large chunks accumulate many matching tokens across multiple topics, scoring highly on BM25 even when only a small portion of the chunk is truly relevant. These false positives enter the RRF pool and compete with genuinely relevant chunks from semantic search. The cross-encoder partially corrects this, but cannot fully recover precision when the input pool is contaminated by oversized, topic-mixed BM25 candidates.

A secondary benefit of hybrid retrieval is reduced total latency for Fixed and Recursive strategies. Despite adding BM25 and reranking stages, total response time drops (e.g., C2's 4,290ms -> C5's 3,061ms, a 29% reduction). This occurs because the reranker selects 5 highly relevant chunks from 15 candidates rather than passing all 15 to the generator, shortening the generation context and therefore generation time.

4.4 Generator Model Comparison: gpt-4o-mini vs. llama-3.1-8b-instant

Pair	Chunking	Mode	GPT Faith.	Llama Faith.	GPT Rel.	Llama Rel.	GPT Total	Llama Total
C1 vs C7	Fixed (A)	Semantic	0.925	0.944 (+2.1%)	0.688	0.703 (+2.2%)	3,073ms	734ms
C2 vs C8	Recursive (B)	Semantic	0.964	0.941 (-2.4%)	0.696	0.702 (+0.9%)	4,290ms	642ms
C3 vs C9	Semantic (C)	Semantic	0.836	0.744 (-11.0%)	0.703	0.696 (-1.0%)	4,509ms	844ms
C4 vs C10	Fixed (A)	Hybrid	0.966	0.965 (-0.1%)	0.714	0.712 (-0.3%)	2,942ms	647ms
C5 vs C11	Recursive (B)	Hybrid	0.982	0.970 (-1.2%)	0.709	0.717 (+1.1%)	3,061ms	619ms
C6 vs C12	Semantic (C)	Hybrid	0.832	0.740 (-11.1%)	0.692	0.663 (-4.2%)	4,096ms	2,849ms

Across configurations using Fixed and Recursive chunking with Hybrid+Rerank retrieval (the highest-quality retrieval setting), gpt-4o-mini and llama-3.1-8b-instant achieve nearly identical faithfulness scores within 0.1-1.2 percentage points. This is the most important finding of the model comparison: when the retrieval pipeline is optimal, the generator's faithfulness bottleneck shifts from model capability to retrieval quality. Both models are capable of synthesising accurate answers when given clean, relevant context.

The performance gap between the two models widens substantially on Semantic chunking. On C6 vs C12, gpt-4o-mini achieves 0.832 faithfulness while llama-3.1-8b-instant drops to 0.740 - an 11.1-point gap. When the retrieved context is noisy or heterogeneous (as with semantic chunking's larger mixed chunks), gpt-4o-mini's stronger instruction-following and larger effective context window allow it to extract relevant information while ignoring irrelevant content. The smaller llama model is more sensitive to retrieval noise and more likely to incorporate information from irrelevant sections into its answers, producing unverifiable claims.

The most striking difference is generation latency. llama-3.1-8b-instant via Groq is approximately 5-7x faster than gpt-4o-mini across comparable configurations (e.g., C5: 2,832ms vs C11: 393ms generation time). For deployment scenarios where latency is a hard constraint, C11 (Recursive + Hybrid + Rerank + llama) represents a strong trade-off: 0.970 faithfulness and 0.717 relevancy at only 619ms total response time.

Corner case - C12 anomaly: llama-3.1-8b-instant on Semantic+Hybrid (C12) has an anomalously high generation time of 2,619ms - far above its 393-604ms range on other configurations. This is caused by semantic chunking's occasionally very large chunks (2,000+ characters) saturating Groq's LPU memory bandwidth when processing long context inputs. This is a chunking-model interaction effect not observed with gpt-4o-mini, which maintains consistent generation speed regardless of context length variations.

4.5 Latency and Computational Efficiency Analysis

Config	Generator	Mode	Ret.(ms)	Gen.(ms)	Total(ms)	Gen% of Total
C1	gpt-4o-mini	Semantic	352	2,721	3,073	88.5%
C2	gpt-4o-mini	Semantic	270	4,020	4,290	93.7%
C3	gpt-4o-mini	Semantic	267	4,243	4,509	94.1%

C4	gpt-4o-mini	Hybrid	239	2,703	2,942	91.9%
C5 (Best)	gpt-4o-mini	Hybrid	229	2,832	3,061	92.5%
C6	gpt-4o-mini	Hybrid	239	3,857	4,096	94.2%
C7	llama-3.1-8b	Semantic	242	493	734	67.2%
C8	llama-3.1-8b	Semantic	240	403	642	62.8%
C9	llama-3.1-8b	Semantic	240	604	844	71.6%
C10	llama-3.1-8b	Hybrid	227	420	647	64.9%
C11	llama-3.1-8b	Hybrid	226	393	619	63.5%
C12 (anomaly)	llama-3.1-8b	Hybrid	229	2,619	2,849	91.9%

Retrieval time is remarkably consistent across all configurations: 226-352ms for all experiments. The slight variation is attributable to Pinecone API network jitter rather than systematic differences between retrieval modes. BM25 scoring runs entirely in-memory in microseconds, and the cross-encoder inference on 15 candidate pairs adds only ~10-20ms, making Hybrid+Rerank effectively cost-free from a latency perspective versus Semantic-Only.

Generation time dominates total latency in all configurations, accounting for 62-94% of total response time. For gpt-4o-mini, generation ranges from 2,703ms (C4) to 4,243ms (C3); for llama-3.1-8b-instant it is consistently below 620ms except for the C12 anomaly. Recursive chunking's smaller average chunk sizes (336 chars) contribute to shorter generation times relative to Semantic chunking (~600 chars), explaining why C2/C5 generation times are lower than C3/C6 for the same model. Optimising the generation model and chunking strategy thus have the largest joint impact on system responsiveness.

4.6 Prompt Engineering Ablation (P1 - P4)

Config	Generation Prompt	Judge Prompt	Faithfulness	Relevancy	Gen.(ms)	Notes
P1 (=C5)	Baseline	Baseline (Binary YES/NO)	0.968	0.695	2,832	Prompt ablation baseline - identical to C5
P2	Improved (CoT+Cite)	Baseline (Binary YES/NO)	1.000	0.749	3,459	Perfect faithfulness; same judge scale as P1
P3	Baseline	Improved (3-Level Rubric)	0.990	0.925	2,958	Improved judge recalibrates both metrics upward
P4	Improved (CoT+Cite)	Improved (3-Level Rubric)	0.997	0.902	4,120	Both improved; relevancy slightly lower than P3

P2 (Improved Generation + Baseline Judge): Introducing chain-of-thought reasoning and mandatory inline citations raises faithfulness from 0.968 to a perfect 1.000 across all 15 test questions, with relevancy improving from 0.695 to 0.749. The CoT instruction encourages the model to explicitly

reason about which retrieved chunks support each claim before committing to an answer, directly reducing hallucination. The mandatory citation format anchors every statement to a specific source chunk. Generation time increases by ~22% (2,832ms -> 3,459ms), reflecting the additional reasoning steps, but no extra API cost for model tier change.

P3 (Baseline Generation + Improved Judge): Switching to the 3-level rubric judge (SUPPORTED / PARTIAL / NOT_SUPPORTED) raises reported relevancy dramatically from 0.695 to 0.925. This should not be interpreted as an absolute improvement in answer quality - the underlying answers are identical to P1. Rather, the improved judge evaluates relevancy directly across four weighted aspects (topic alignment 35%, specificity 25%, scope 20%, completeness 20%), each scoring more accurately than the binary embedding-cosine proxy in the baseline. The baseline's question-generation approach systematically underestimates relevancy when the answer is complete but uses different terminology than the original query; the aspect-based judge corrects this bias.

P4 (Both Improved): Combining both improved prompts achieves 0.997 faithfulness and 0.902 relevancy. The slight relevancy decrease from P3 (0.925) to P4 (0.902) is noteworthy: the improved generation prompt's structured [Reasoning]/[Answer] output format occasionally results in shorter, more focused Answer sections that score lower on the completeness aspect of the improved judge, even though the answer is more precisely faithful. This is a known design trade-off: more constrained generation prompts improve faithfulness at the potential cost of answer comprehensiveness.

Important caveat on cross-comparison: P1-P2 and P3-P4 use different judge scales and are not directly comparable on raw numbers. The correct interpretation is: improving the generation prompt (P1->P2) delivers a genuine faithfulness gain measurable on any judge; improving the judge (P1->P3) delivers a more accurate measurement of existing quality rather than an absolute quality improvement.

4.7 Cross-Dimensional Analysis and Key Findings

4.7.1 Faithfulness vs. Relevancy Trade-off

Faithfulness and relevancy are weakly positively correlated but not redundant across configurations. C5 achieves the highest faithfulness (0.982) but not the highest relevancy (0.709 vs C4's 0.714 or C11's 0.717). Semantic chunking configurations consistently show slightly higher relevancy than Fixed chunking on comparable setups, despite lower faithfulness. This reflects the independence of the two metrics: faithfulness measures claim-context grounding, while relevancy measures whether the answer addresses the question at all. A highly faithful but partially off-topic answer scores high on faithfulness and low on relevancy; a relevant but sometimes hallucinated answer does the opposite. Both metrics must be considered jointly when selecting a best configuration.

4.7.2 Faithfulness Variance as a Quality Signal

Per-question faithfulness scores reveal that variance is as informative as the mean. Semantic chunking configurations (C3, C6, C9, C12) exhibit the highest per-question variance, with individual faithfulness scores spanning from near 0.0 to 1.0 within the same configuration. C6 records per-question scores as low as 0.125 and 0.455 alongside perfect 1.0 scores on other questions. By contrast, C5 (Recursive + Hybrid) records only one question below 0.9 (Q10 = 0.700), indicating robust, consistent performance. High variance is undesirable in production systems as it means

unpredictable quality across different query types. Consistency is a strong additional argument for Recursive chunking beyond its mean faithfulness advantage.

4.7.3 Chunking Strategy Is the Dominant Performance Driver

Comparing effect sizes across all controlled experiments: switching from Semantic (C) to Recursive (B) chunking under identical retrieval and generation settings improves faithfulness by 12.8-14.8 percentage points (C3->C2: 0.836->0.964; C6->C5: 0.832->0.982). By comparison, switching from Semantic-Only to Hybrid+Rerank retrieval improves faithfulness by only 0.4-4.1 percentage points. Switching from llama to gpt-4o-mini on well-retrieved contexts improves faithfulness by only 0.1-1.2 points. Chunking strategy is therefore the single highest-impact design decision in this RAG system - larger than retrieval mode or model choice combined. Teams optimising a RAG pipeline should prioritise chunking strategy as the primary hyperparameter.

4.7.4 Interaction Effects: Chunking x Retrieval x Model

The configurations form a 3x2x2 factorial design (3 chunking strategies, 2 retrieval modes, 2 generator models). The most important interaction is between chunking strategy and retrieval mode on Semantic chunking: while Fixed and Recursive chunking consistently benefit from hybrid retrieval, Semantic chunking slightly degrades. This reveals that the hybrid pipeline is optimised for moderately-sized, topically coherent chunks and becomes counterproductive when chunks are large and topic-mixed. The interaction between model and chunking is also significant: the llama model's performance degrades more steeply than gpt-4o-mini when retrieval quality drops (on Semantic chunking, the gap is 11 percentage points vs 1-2 points on better chunking), indicating that smaller models require higher-quality retrieval to achieve comparable faithfulness. These interaction effects demonstrate that optimal RAG design requires joint optimisation of all three components.

4.7.5 Cost-Quality Trade-off Summary

Configuration	Faithfulness	Relevancy	Total Latency	API Cost	Recommendation
C5: Recursive+Hybrid+GPT	0.982	0.709	3,061ms	Paid (OpenAI)	Best quality - use when faithfulness is critical
C11: Recursive+Hybrid+Llama	0.970	0.717	619ms	Free (Groq)	Best value - 5x faster, minimal quality loss
P2: C5 + CoT+Cite prompt	1.000	0.749	3,688ms	Paid (OpenAI)	Best faithfulness - add CoT prompt at no model cost
C4: Fixed+Hybrid+GPT	0.966	0.714	2,942ms	Paid (OpenAI)	Good balance if Recursive chunking not available

For maximum answer quality with the baseline judge, C5 (Recursive+Hybrid+gpt-4o-mini) at 0.982 faithfulness and 0.709 relevancy is the best configuration. For speed and cost efficiency, C11 (Recursive+Hybrid+llama-3.1-8b-instant) achieves 0.970 faithfulness and 0.717 relevancy at 619ms total and zero API cost - the quality difference versus C5 is only 1.2pp faithfulness. For the best possible faithfulness with no additional model upgrade, applying the improved CoT+Citation generation prompt (P2) to the C5 pipeline achieves perfect 1.000 faithfulness at a modest 20% latency increase.

4.8 Example Queries: Extracted Claims and Verification Results

The following three examples are drawn from the best-performing configuration, C5 (Recursive chunking + Hybrid + Rerank + gpt-4o-mini, T=0.1). For each query the table lists every claim extracted from the generated answer and the binary YES/NO verdict returned by the gpt-4o-mini judge. ✅ = SUPPORTED, ❌ = NOT SUPPORTED. The Faithfulness score is the proportion of supported claims; the Relevancy score is computed via the Q-Gen + Cosine method described in Section C.

4.8.1 Query 1: Nested Learning and In-Context Learning

Query: How does the concept of Nested Learning relate to the idea of in-context learning in large models, and what implications does this have for the design of more expressive learning algorithms?

Generated Answer (C5): The concept of Nested Learning (NL) relates to in-context learning (ICL) by providing a framework that represents machine learning models as a set of nested, multi-level, and/or parallel optimization problems, each with its own context flow. This perspective suggests that existing deep learning methods learn from data by compressing their context flow, which allows ICL to naturally emerge in large models. The implications for the design of more expressive learning algorithms include the potential for higher-order in-context learning and enhanced continual learning capabilities. NL advocates for the development of more expressive optimizers and self-modifying learning modules, which can lead to improved performance in tasks such as language modeling, knowledge incorporation, and few-shot generalization.

Sources Retrieved: Nested Learning: The Illusion of Deep Learning Architectures (x4); Learning-to-Learn at Scale: Promises and Challenges (x1)

#	Extracted Claim	Verdict
1	NL relates to in-context learning (ICL) by providing a framework that represents ML models as nested, multi-level, and/or parallel optimization problems.	✅ YES
2	NL provides a framework for understanding how models can adapt and learn from their context.	✅ YES
3	NL involves a set of nested, multi-level, and/or parallel optimization problems.	✅ YES
4	Existing deep learning methods compress their own context flow.	✅ YES
5	In-context learning can naturally emerge in large models.	✅ YES
6	ICL capabilities can be unified across different model architectures and objectives.	✅ YES
7	The connection between NL and ICL can be viewed through the lens of associative memory.	✅ YES
8	More expressive learning algorithms should not rely on traditional test/training phases.	✅ YES
9	The nested structure of NL models enhances their ICL capabilities.	✅ YES
10	NL suggests that higher-order in-context learning can improve model performance in continual learning tasks.	✅ YES

#	Extracted Claim	Verdict
11	There is a need for innovative design approaches that leverage nested systems.	✅ YES
	Result: 12/12 claims supported — Faithfulness = 1.000 | Relevancy = 0.721	

4.8.2 Query 2: Nested Optimization Problems and Context Flow

Query: What is the significance of representing a machine learning model as a set of nested, multi-level, and/or parallel optimization problems in the context of Nested Learning, and how does this relate to the concept of 'context flow'?

Generated Answer (C5): The significance of representing a machine learning model as a set of nested, multi-level, and/or parallel optimization problems in the context of Nested Learning (NL) lies in its ability to enhance the expressiveness and capability of learning algorithms. This representation allows each component of the model to have its own internal gradient flow on its own context, facilitating a more coherent and interconnected system of optimization processes. The concept of 'context flow' refers to how information is processed and utilized within these nested optimization problems. By structuring the learning process in this way, NL aims to unlock higher-order in-context learning and improve continual learning capabilities, thereby addressing fundamental challenges in machine learning models regarding continual learning, self-improvement, and effective solution finding.

Sources Retrieved: Nested Learning: The Illusion of Deep Learning Architectures (x4); Dynamic Nested Hierarchies: Self-Evolution in ML for Lifelong Intelligence (x1)

#	Extracted Claim	Verdict
1	Representing a ML model as nested, multi-level, and/or parallel optimization problems enhances the expressiveness and capability of learning algorithms.	✅ YES
2	Each component of the model in NL has its own internal gradient flow on its own context.	✅ YES
3	The representation in NL facilitates a more coherent and interconnected system of optimization processes.	✅ YES
4	'Context flow' refers to how information is processed and utilized within each nested optimization problem.	✅ YES
5	Existing deep learning methods in NL learn from data by compressing their own context flow.	✅ YES
6	Compressing context flow leads to the emergence of in-context learning in large models.	✅ YES
7	NL aims to unlock higher-order in-context learning.	✅ YES
8	NL aims to improve continual learning capabilities.	✅ YES
9	NL addresses fundamental challenges regarding continual learning, self-improvement, and effective solution finding.	✅ YES

#	Extracted Claim	Verdict
	Result: 9/9 claims supported — Faithfulness = 1.000 | Relevancy = 0.681	

4.8.3 Query 3: Fast Weight Programmers vs. Conventional RNNs

Query: How do Fast Weight Programmers (FWPs) differ from conventional Recurrent Neural Networks (RNNs) in terms of their hidden state representation, and what implications does this have for their ability to store short-term memory?

Generated Answer (C5): Fast Weight Programmers (FWPs) differ from conventional Recurrent Neural Networks (RNNs) in that FWPs utilize two-dimensional (2D) matrix-form hidden states, while conventional RNNs use vector-form hidden states. This matrix-form representation in FWPs allows for dynamically changing synaptic weights, which can be interpreted as time-varying synaptic weights that maintain short-term memory. In contrast, conventional RNNs have fixed synaptic weights after training, limiting their ability to adaptively store and utilize short-term memory. This unique property of FWPs enhances their relevance in both machine learning and computational neuroscience, as it offers a more biologically compatible approach to local learning and memory retention.

Sources Retrieved: Fast Weight Programming and Linear Transformers: From ML to Neurobiology (x4); Nested Learning: The Illusion of Deep Learning Architectures (x1)

#	Extracted Claim	Verdict
1	FWPs utilize two-dimensional (2D) matrix-form hidden states.	✅ YES
2	Conventional RNNs use vector-form hidden states.	✅ YES
3	The matrix-form representation in FWPs allows for dynamically changing synaptic weights.	✅ YES
4	FWPs can maintain short-term memory through time-varying synaptic weights.	✅ YES
5	Conventional RNNs have fixed synaptic weights after training.	✅ YES
6	Fixed synaptic weights in conventional RNNs limit their ability to adaptively store and utilize short-term memory.	✅ YES
7	FWPs enhance their relevance in machine learning and computational neuroscience.	✅ YES
8	FWPs offer a more biologically compatible approach to local learning and memory retention.	✅ YES
	Result: 8/8 claims supported — Faithfulness = 1.000 | Relevancy = 0.695	

All three examples above are from configuration C5 and reflect the consistency of the pipeline: across 29 total claims extracted, 29 were verified as supported, yielding a perfect Faithfulness score of 1.000 for these examples. Relevancy scores range from 0.681 to 0.721, reflecting moderate semantic alignment between generated answers and the original queries under the Q-Gen + Cosine method. These examples are representative of C5's typical behaviour on well-retrieved, factually grounded questions; per-question variance for C5 is the lowest of any configuration tested, with only one question falling below 0.9 across the full 30-question golden set.

5. Best Model Selection

5.1 Selection Framework

Selecting the best configuration requires balancing four measurable dimensions evaluated across all 19 configurations in Section D: **faithfulness** (are the generated claims grounded in retrieved context?), **relevancy** (does the answer address the original query?), **latency** (total end-to-end response time), and **consistency** (low per-question variance in faithfulness scores). A configuration that achieves a high mean faithfulness but produces catastrophically low scores on individual questions is not reliable enough for a production RAG system. These four criteria, weighed together, form the basis for the selection below.

5.2 Primary Best Configuration: C5 (Recursive + Hybrid + Rerank + gpt-4o-mini)

C5 is selected as the primary best configuration. It achieves the highest faithfulness score among all main ablation configurations (0.982), the second-highest relevancy in the main ablation group (0.709), a competitive total latency of 3,061ms, and — critically — the lowest faithfulness variance of any configuration tested, with only one question falling below 0.9 (Q10 = 0.700) across the entire golden set.

Faithfulness justification. The 0.982 faithfulness score is the direct result of two compounding advantages. First, Recursive chunking (B) produces structure-preserving chunks with an average size of 336 characters — smaller and more topically coherent than Fixed (497 chars) or Semantic (~600 chars) alternatives. These coherent units allow the generator to map every produced claim cleanly back to a single chunk, reducing hallucination opportunities. Second, Hybrid + Rerank retrieval combines BM25's exact-term matching with dense semantic retrieval, then filters the joint candidate pool with a cross-encoder reranker. The cross-encoder's joint query–chunk scoring benefits directly from Recursive chunking's coherent information units, producing the cleanest possible input context for the generator. The data confirms this synergy: Fixed chunking gains 4.4 percentage points moving from Semantic-Only to Hybrid (C1→C4), while Recursive gains only 1.8 points (C2→C5) — not because Hybrid helps it less, but because it already started closer to ceiling.

Relevancy justification. C5's relevancy score of 0.709 is slightly below C4's 0.714 and C11's 0.717. This 0.005–0.008 point gap is not a meaningful quality difference — it falls within the natural variance of the Q-Gen + Cosine relevancy judge, which is sensitive to lexical alignment between the generated answer and re-generated question. What matters more is that C5 achieves its relevancy score alongside the highest faithfulness, meaning it is not trading one metric for the other. Configurations that sacrifice faithfulness to gain relevancy points (such as Semantic chunking variants C3 and C6) are not preferable: an answer that is topically on-target but contains unsupported claims is less trustworthy than one that is slightly narrower in scope but fully grounded.

Consistency justification. Per-question faithfulness variance is a critical but often overlooked production metric. Semantic chunking configurations (C3, C6, C9, C12) all exhibit per-question faithfulness scores ranging from near 0.0 to 1.0 within the same configuration — C6 records individual scores as low as 0.125 and 0.455. This unpredictability means the system fails badly on some query types while excelling on others, which is unacceptable in a deployed system. C5 records only one question below 0.9 across all 15 golden set questions, making it robustly reliable across the full range of query types present in the corpus. High consistency, combined with high mean performance, is the decisive advantage of C5 over all competing configurations.

Latency justification. C5's total latency of 3,061ms is the result of a counterintuitive finding: adding BM25 and cross-encoder reranking to the retrieval pipeline actually *reduces* total latency compared to Semantic-Only (C2: 4,290ms → C5: 3,061ms, a 29% reduction). The reranker narrows 15 retrieval candidates down to the 5 most relevant before generation, shortening the generator's context window and directly reducing generation

time — the dominant cost component at 88–94% of total latency. The additional BM25 + reranking cost is negligible (≤20ms). C5 therefore achieves both higher quality and lower latency than its Semantic-Only equivalent C2, making the performance case unambiguous.

5.3 Enhanced Best Configuration: P2 (C5 + CoT+Citation Prompt)

If the evaluation criterion is purely maximizing faithfulness — the most critical metric in a research-corpus RAG system where unsupported claims are directly harmful — then P2 is the best configuration. P2 applies the improved Chain-of-Thought + mandatory inline citation generation prompt to the C5 pipeline, achieving a perfect faithfulness score of 1.000 across all 15 golden set questions and a relevancy of 0.749, the highest relevancy score recorded under the baseline judge scale.

The improvement from C5 to P2 is achieved at zero additional model cost — the same gpt-4o-mini model is used, with only the generation prompt modified. The trade-off is a 20% latency increase (3,061ms → 3,688ms) attributable to the additional CoT reasoning steps before the final answer is produced. For a research-assistant application, where answer trustworthiness outweighs a 627ms latency increment, this trade-off is clearly worthwhile. P2's CoT instruction forces the model to explicitly reason through which retrieved chunks support each claim before committing to an answer, while the mandatory citation format anchors every statement to a verifiable source chunk — a structural guarantee that the baseline prompt cannot provide.

It is important to note that P2's perfect faithfulness score is measured under the binary YES/NO judge inherited from C5. The prompt ablation in Section D.6 confirms that the binary judge systematically underestimates relevancy (it scores the same answers at 0.695–0.749 that the improved aspect-decomposed judge rates at 0.902–0.925). This means P2's true relevancy quality is almost certainly higher than 0.749 — a key reason why it is recommended as the enhanced best configuration rather than a marginal incremental upgrade.

5.4 Speed-Optimised Alternative: C11 (Recursive + Hybrid + Rerank + llama-3.1-8b-instant)

For deployment contexts where latency or API cost are hard constraints, C11 is the recommended configuration. It uses the same Recursive + Hybrid + Rerank retrieval pipeline as C5 but substitutes llama-3.1-8b-instant (served via Groq's LPU infrastructure) for gpt-4o-mini as the generator. The resulting faithfulness of 0.970 and relevancy of 0.717 represent only a 1.2 percentage point faithfulness reduction versus C5, while the total response time drops from 3,061ms to 619ms — a 5x speed improvement — at zero API cost.

The generator comparison in Section D.4 demonstrates that the quality gap between gpt-4o-mini and llama-3.1-8b-instant is retrieval-dependent: on well-retrieved contexts (Fixed or Recursive chunking with Hybrid retrieval), the two models are nearly indistinguishable on faithfulness (within 0.1–1.2 points). The performance bottleneck shifts from model capability to retrieval quality when the pipeline is well-configured, meaning a smaller model can match a larger one's output fidelity given clean, topically coherent context. C11 exploits this finding directly: its retrieval pipeline is optimal, so the generator's size limitation does not materially degrade answer quality.

E.5 Configurations Ruled Out and Why

Semantic chunking variants (C3, C6, C9, C12). Despite Semantic chunking's intuitive appeal — splitting on topic boundaries rather than character counts — all four Semantic configurations are eliminated on faithfulness grounds. The best Semantic result (C3: 0.836 and C6: 0.832) is 14.6–15.0 percentage points below C5, the largest performance gap observed across any single design decision in the entire experiment set. The root cause is chunk size heterogeneity: bimodal chunk distributions produce diluted embedding vectors for long, topic-mixed chunks, degrading retrieval precision and introducing noise into the generation context. The faithfulness variance problem (individual scores as low as 0.125 in C6) makes these configurations unreliable regardless of their mean score.

Fixed chunking with Semantic-Only retrieval (C1). C1 achieves only 0.925 faithfulness — the lowest among gpt-4o-mini configurations — and has the highest retrieval latency at 352ms due to Pinecone query overhead without BM25 pre-filtering. The combination of structural mid-sentence

splits and purely semantic retrieval produces the weakest retrieval pool in the experiment set. C1 is retained in the report as the baseline reference point but is not competitive against any Hybrid retrieval configuration.

Llama-as-Judge (C13). C13 was included to validate judge robustness rather than as a candidate best configuration. Using llama-3.1-8b-instant as both the judge and the generator on the same pipeline as C5 confirmed that the evaluation pipeline is judge-stable — the scores produced by the llama judge were directionally consistent with gpt-4o-mini's judgments — but the llama judge is excluded from production recommendation because its instruction-following on structured JSON rubrics is less reliable than gpt-4o-mini's, introducing evaluation noise that could misrepresent actual answer quality.

High-temperature variants (C14, C15). Temperature 0.7 ablations confirmed that increased generation randomness does not improve faithfulness or relevancy on this task. Both C14 (GPT at T=0.7) and C15 (Llama at T=0.7) show no consistent metric improvement over their T=0.1 counterparts, while introducing unnecessary stochastic variation into answer content. For a factual research-corpus RAG system, low temperature (T=0.1) is the correct setting, and all recommended configurations use it.

5.6 Summary Recommendation

The experiments across 19 configurations converge on a clear and reproducible conclusion: **chunking strategy is the dominant design decision in this RAG system**, with Recursive chunking outperforming all alternatives by a wider margin than any other hyperparameter swap. Given Recursive chunking, the optimal downstream pipeline is Hybrid + Rerank retrieval and gpt-4o-mini generation at T=0.1, giving configuration C5. Applying the CoT+Citation generation prompt to C5 yields P2, which further raises faithfulness to a perfect 1.000 with no model-tier cost. For latency-constrained scenarios, C11 (substituting llama-3.1-8b-instant for gpt-4o-mini) delivers nearly equivalent quality at 5x the speed and zero API cost.

These three recommended configurations — C5, P2, and C11 — are not competing choices but a deployment menu: C5 for the quality baseline, P2 for maximum faithfulness, and C11 for production at scale. All three share the same retrieval foundation, meaning the core pipeline investment is identical and only the generation layer changes across the three options.

6. Reproducibility:

This section provides all information necessary to fully replicate the system, from corpus collection through to final evaluation. The goal is that any researcher with access to the referenced APIs and platforms should be able to reproduce the results reported in Section 4 without ambiguity.

6.1 Hardware and Platform Requirements

- Kaggle with T4 GPU (16 GB VRAM) — required for Semantic Chunking (Strategy C) and document embedding
- Hugging Face Spaces (free tier, CPU) — for the live web deployment
- Pinecone Free Starter tier — three separate indexes required (one per chunking strategy)
- OpenAI API key — for text-embedding-3-small (embedding) and gpt-4o-mini (generation and judging)
- Groq API key (free) — for llama-3.1-8b-instant and llama-3.3-70b-versatile

6.2 Corpus Collection

1. Navigate to the anchor paper on Google Scholar: Nested Learning: The Illusion of Deep Learning Architectures (Behrouz et al., NeurIPS 2025) — arXiv:2512.24695.
2. Click "Cited by" to retrieve all citing papers at time of collection. Download all PDFs accessible via arXiv, Springer, ResearchGate, TechRxiv, or OpenReview.
3. Apply the curation criteria described in Section 2.3 to select 15 papers. The exact 15 papers used are listed in Table 2 of this report with their arXiv/source links.

6.3 Preprocessing Steps (Notebook: Preprocessing.ipynb)

4. Load each PDF using PyMuPDF (pip install pymupdf). Extract raw text page by page.
5. Remove reference sections: identify first match of regex r'^(References|REFERENCES|Bibliography)\s*$' and strip everything following.
6. Strip isolated page-number lines: remove lines matching r'^\d+$'.
7. Collapse multiple blank lines: replace 3+ consecutive blank lines with 2.
8. Remove decorative dividers: strip lines matching r'^[-_=]{3,}$'.
9. Save cleaned output as UTF-8 .txt files (one per paper).

6.4 Chunking Configuration

- Strategy A (Fixed): CharacterTextSplitter(chunk_size=512, chunk_overlap=50, separator='\n')
- Strategy B (Recursive): RecursiveCharacterTextSplitter(chunk_size=400, chunk_overlap=80)

- Strategy C (Semantic): SemanticChunker(embeddings=all-MiniLM-L6-v2, breakpoint_threshold_type='percentile', breakpoint_threshold_amount=70, min_chunk_size=80)

6.5 Embedding and Pinecone Setup

10. Create three Pinecone indexes (free tier) with dimension=1536, metric='cosine'. Name them e.g. rag-fixed, rag-recursive, rag-semantic.
11. For each chunking strategy, embed all chunks using openai.embeddings.create(model='text-embedding-3-small').
12. Upsert all chunk embeddings to the corresponding Pinecone index in batches of 100. Store chunk text and source paper ID as metadata.

6.6 BM25 Index Construction

13. Install rank-bm25 (pip install rank-bm25).
14. For each chunking strategy, tokenise all chunks using lowercase + whitespace split: corpus_tokenised = [chunk.lower().split() for chunk in chunks].
15. Build the BM25 index: bm25 = BM25Okapi(corpus_tokenised). Persist the chunked corpus list alongside the index for lookup.

6.7 Golden Set Generation

16. For each of the 15 papers, concatenate the first 4 chunks (abstract + introduction).
17. Pass to llama-3.3-70b-versatile via Groq API with temperature=0.2 and the exact golden set prompt shown in Figure 13.
18. Parse the JSON output and discard Q&A pairs where question length < 20 chars or answer length < 50 chars.
19. Target: 30 high-quality Q&A pairs distributed across the 15 papers.

6.8 Retrieval and Generation Pipeline

20. Embed query using text-embedding-3-small.
21. Run BM25.get_scores(query_tokens) — take top 15 by score.
22. Run Pinecone index.query(vector=query_embedding, top_k=15) — take top 15 by cosine similarity.
23. Merge using RRF with k=60: for each unique chunk, sum 1/(k+rank) across both lists.
24. Sort merged pool by RRF score descending; take top 15.
25. Pass top-15 (query, chunk) pairs to CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2').predict(); sort descending; take top 5.
26. Pass top-5 chunks to generator with the exact prompt shown in Figure 5 (baseline) or Figure 6 (improved), temperature=0.1.

6.9 Evaluation Pipeline

Faithfulness: (1) Extract claims using the prompt in Figure 7 (or Figure 9 for improved). (2) For each claim, verify against the top-5 context using the prompt in Figure 8 (or Figure 10 for improved). (3) Compute Faithfulness = (supported claims) / (total claims).

Relevancy: (1) Generate 3 surrogate questions from the answer using the prompt in Figure 11 (or Figure 12 for improved). (2) Embed each surrogate question and the original query using text-embedding-3-small. (3) Compute Relevancy = mean cosine similarity across the 3 pairs.

References

[1] Behrouz, A., Razaviyayn, M., Zhong, P., Mirrokni, V.: Nested Learning: The Illusion of Deep Learning Architectures. In: The 39th Annual Conference on Neural Information Processing Systems (NeurIPS 2025). https://arxiv.org/pdf/2512.24695

[2] Behrouz, A., Li, J., Deng, J., Zhong, P., Razaviyayn, M., Mirrokni, V.: Memory Caching: RNNs with Growing Memory (2026). https://arxiv.org/pdf/2602. 24281

[3] Meskin, D., Mirrokni, V., Najar, A., Behrouz, A.: HYDRA: Dual Exponentiated Memory for Multivariate Time Series Analysis (2025). https://arxiv.org/pdf/ 2511.00989

[4] Wang, X. et al.: Beyond Test-Time Training: Learning to Reason via HardwareE cient Optimal Control (2026). https://arxiv.org/pdf/2603.09221

[5] Tandon, A. et al.: End-to-End Test-Time Training for Long Context (2025). https: //arxiv.org/pdf/2512.23675

[6] Jafari, A.A., Ozcinar, C., Anbarjafari, G.: Dynamic Nested Hierarchies: Pioneering Self-Evolution in ML Architectures for Lifelong Intelligence (2025). https://arxiv. org/pdf/2511.14823

[7] Cao, Z., Cao, Y., Wu, J.: CRAD-HOPE: Brain-Inspired Nested Learning Framework for Few-Shot Anomaly Detection (2026).

[8] Irie, K., Gershman, S.: Fast Weight Programming and Linear Transformers: From Machine Learning to Neurobiology (2026). https://arxiv.org/pdf/2508.08435

[9] Zahn, M., Beton, J., Chana, S.: Attention Is Not Retention: The Orthogonality Constraint in In nite-Context Architectures (2025).

[10] Gosmar, D., Dah, K.: Prompt Injection Mitigation with Agentic AI, Nested Learning, and AI Sustainability via Semantic Caching (2026). https://arxiv.org/pdf/2601. 13186

[11] Sun, Y. et al.: Vision Transformers That Never Stop Learning (2026). https:// arxiv.org/pdf/2603.07787

[12] Wang, Z. et al.: AllMem: A Memory-Centric Recipe for E cient Long-Context Modeling (2026). https://arxiv.org/pdf/2602.13680

[13] Chen, Y. et al.: MSA: Memory Sparse Attention for E cient End-to-End Memory Model Scaling to 100M Tokens (2026). https://arxiv.org/pdf/2603.23516

[14] Rauba, P., Fanconi, C., van der Schaar, M.: Tiny Autoregressive Recursive Models (2026). https://arxiv.org/pdf/2603.08082

[15] Wang, Y. et al.: Learning-to-Learn at Scale: Promises and Challenges (2026).

[16] Robertson, S., Zaragoza, H.: The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval 3(4), 333 389 (2009).

[17] Cormack, G., Clarke, C., Buettcher, S.: Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In: SIGIR 2009.

[18] Nogueira, R., Cho, K.: Passage Re-ranking with BERT. arXiv:1901.04085 (2019).

[19] Reimers, N., Gurevych, I.: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.